Supervyse OPF Web UI Developer's Guide

Revision 1.0

Aug 2024

Copyright (c) 2025, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Content

1. Introduction 5
2. Customize a New Web Page 6
   - 2.1 Frontend 6
     - 2.1.1 Add Menu Items 6
     - 2.1.2 Create and Integrate a New Page Component 8
     - 2.1.3 Add the Appropriate Language Strings 10
     - 2.1.4 Add Necessary Stores for State Management 12
   - 2.2 Backend 14
     - 2.2.1 CGI Source Code 14
     - 2.2.2 Build 15
     - 2.2.3 Deploy 15
   - 2.3 Verify 16
   - 2.4 Customize Page Visibility 18
3. Customize a New Language 20
   - 3.1 Language Key Fallback Policy 21
4. Customize the Web Logo 23
   - 4.1 Replace the Logos 24
   - 4.2 Reference 28
5. Customize the Menu Style 30
   - 5.1 Enable Menu Style 31
   - 5.2 Configure Menu Style 32
   - 5.3 Enable Menu Style for One-Time Use at Runtime 34
   - 5.4 Enable Normal/Demo Mode Toggling 35
6. Develop and Debug 38
   - 6.1 Develop with the Vue Dev Server 38
     - 6.1.1 Install the Prerequisites 38
     - 6.1.2 Configure Vue Dev Server 38
     - 6.1.3 Run the Vue Dev Server 39
   - 6.2 Debug with Vue.js Devtools 41
     - 6.2.1 How to Use Vue.js Devtools 42
     - 6.2.2 Example to Inspect a Component and Vuex Store Using Vue.js Devtools 43
7. Vue with ESLint 45
   - 7.1 How to Use ESLint Verify 45
   - 7.2 How to Config ESLint Rules 46
   - 7.3 How to Ignore ESLint Rules 47
8. Use Browserslist to Support Specific Browser Versions 49

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2023-03-20	Chijung Lin	Chijung Lin	Create this document provide a guide for web UI customization (included change menu item and change logo/resources/..).
1.1	2023-06-30	Chijung Lin	Chijung Lin	Review content for change menu style.
1.2	2023-07-06	Jamey Hou	Chijung Lin	Add customization menu style.
1.3	2024-03-25	Chijung Lin	Chijung Lin	Update the sample case for changing logo.
1.4	2024-11-11	Jamey Hou	Ethan Tesng	Readjust the entire document.
1.5	2025-05-15	Jamey Hou	Ethan Tesng	Readjust the entire document.

1. Introduction

Welcome to the Supervyse OPF Web UI Developer's Guide! This guide is designed to help you navigate the process of creating, customizing, and optimizing web interfaces for the Supervyse OPF. Whether you are building a new web page, customizing the web logo, adjusting the menu style, or managing the visibility of pages, this guide will provide you with the knowledge and tools to get the job done efficiently.

This guide covers the following key topics to help you customize and enhance your web application:

- **Customize a New Web Page**
  Learn how to create and integrate new pages to extend your web application's functionality, including configuring page visibility.
- **Customize a New Language**
  Add new language options to support multilingual user interfaces.
- **Customize the Web Logo**
  Update the web application’s logo to reflect your brand identity.
- **Customize the Menu Style**
  Modify the appearance and layout of the menu to align with your design preferences.
- **Develop and Debug**
  Use efficient development practices and debugging tools to ensure smooth functionality.
- **Vue with ESLint**
  Maintain high-quality code with ESLint configurations tailored for Vue.js projects.
- **Browserslist**
  Support Specific Browser Versions.

This guide assumes you have basic knowledge of web development, including **HTML**, **CSS**, **JavaScript**, and the **Vue.js** framework. By the end of this guide, you will be able to improve the Supervyse OPF Web UI, create web pages, and deliver a unique user experience tailored to your organization’s needs.

Let’s get started on creating powerful, personalized web interfaces with Supervyse OPF!

2. Customize a New Web Page

This section provides a guide on how to create a new web page and integrate it into the web menu using the Vue framework. It also covers how to toggle page visibility, offering you flexible control over page accessibility and efficient page management.

2.1 Frontend

To add a new page in the frontend, follow these steps:

- Add a menu item.
- Create and integrate a new page component.
- Include relevant language strings.
- Add the required stores for state management. Refer to Section 2.1.4.

2.1.1 Add Menu Items

Modify the following file and add menu-related settings.
Location: isc-src/web/webui-vue/src/env/insyde/json/menu.json
Insert the following code under "logs" -> "children" section (see preview below).

```
diff --git a/web/webui-vue/src/env/insyde/json/menu.json b/web/webui-vue/src/env/insyde/json/menu.json
index 02edae4e5b..64c6d548f8 100644
--- a/web/webui-vue/src/env/insyde/json/menu.json
+++ b/web/webui-vue/src/env/insyde/json/menu.json
@@ -25,20 +25,30 @@
             "CONFIG_WEB_PAGE_CFG_SYSTEM_DASHBOARD"
         ]
     },
     "logs":
     {
         "index": 3,
         "label": "appNavigation.logs",
         "icon": "iconLogs",
         "children":
         {
+            "test-logs":
+            {
+                "index": 1,
+                "label": "appNavigation.Test",
+                "route": "/logs/test",
+                "component": "@/views/Logs/Test",
+                "view": [2, 3, 4],
+                "change": [3, 4],
+                "depends": []
+            },
             "event-logs":
             {
                 "index": 1,
                 "label": "appNavigation.eventLogs",
                 "route": "/logs/event-logs",
                 "component": "@/views/Logs/EventLogs",
                 "depends": [
                     "CONFIG_OFD_PAGE_EVENT_LOGS"
                 ]
             },
```

Figure 2.1.1-1 Diff tool comparison of menu.json

```
            "test-logs":
```

```
        {
            "index": 1,
            "label": "appNavigation.Test",
            "route": "/logs/test",
            "component": "@/views/Logs/Test",
            "view": [2, 3, 4],
            "change": [3, 4],
            "depends": []
        },
```

Field Description:

- **test-logs**: A unique identifier for this element, which can be any non-repeating string compared to sibling elements.
- **index**: Defines the menu order (items with lower numbers appear first; items with the same number are displayed in the order they are listed).
- **label**: Specifies the menu name and page title.
- **route**: Displays the path in the URL.
- **component**: The location of the page component.
  Note: The '@' symbol corresponds to the 'src' path and must be used as '@' here.
- **view**: Permission to view the page.
  Roles associated with this permission: 4: Administrator, 3: Operator, 2: ReadOnly.
- **change**: Permission to modify the page.
  Roles associated with this permission: 4: Administrator, 3: Operator, 2: ReadOnly.
- **disable**: Page Visibility: The default value is false.
  (When set to true, the page will be hidden.)
- **depends**: Dependency on Settings: The page will be hidden if the necessary settings are not met. This feature enables toggling page visibility through menuconfig, providing greater control over which pages are accessible.

The preview is shown in the image below. Since the page component has not been implemented yet, a 'page not found' message will be displayed.

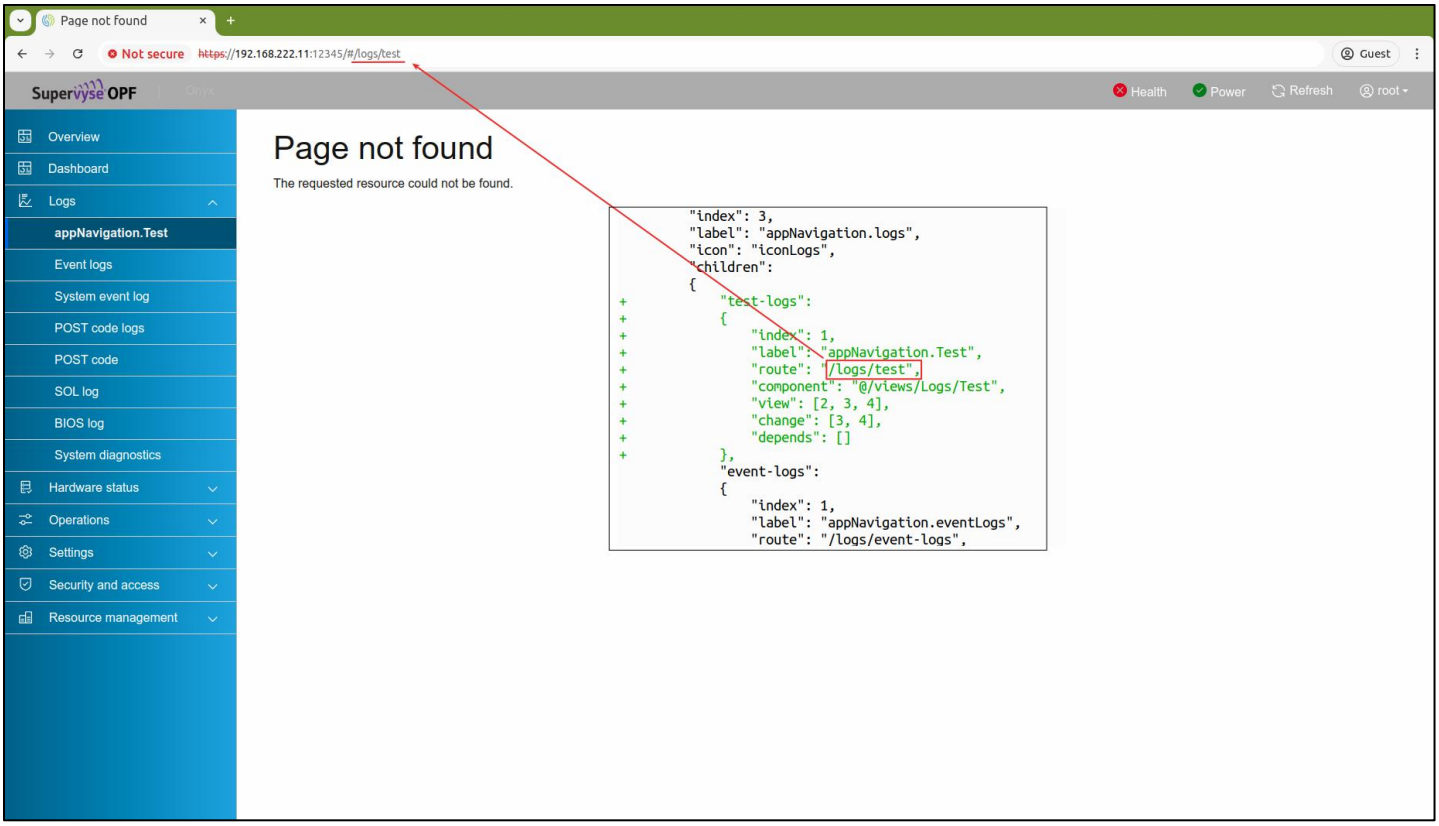

Figure 2.1.1-2 Preview

2.1.2 Create and Integrate a New Page Component

This section will guide you through creating a new page, Test.vue, that includes a message and two buttons. The message content will be updated upon clicking either of the buttons.

The following is an example of how to modify it.

Location: isc-src/web/webui-vue/src/views/Logs/Test/index.js

```
import Test from './Test.vue';
export default Test;
```

Location: isc-src/web/webui-vue/src/views/Logs/Test/Test.vue

```
<template>
  <b-container fluid="xl">
    <page-title />
    <b-row>
      <b-col>
        <div>{{ `${title}: ${message}` }}</div>
      </b-col>
    </b-row>
    <b-row>
      <b-col>
```

```
        <b-button variant="primary" class="mr-1" @click="onSave">
          {{ $t('global.action.save') }}
        </b-button>
        <b-button variant="secondary" class="mr-1" @click="onReset">
          {{ $t('global.action.reset') }}
        </b-button>
      </b-col>
    </b-row>
  </b-container>
</template>

<script>
import PageTitle from '@/components/Global/PageTitle';

export default {
  components: {
    PageTitle,
  },
  data() {
    return {
      title: 'Message',
    };
  },
  computed: {
    message() {
      return this.$store.getters['mytest/message'];
    },
  },
  created() {
    this.startLoader();
    // Do something
    this.endLoader();
  },
  methods: {
    onSave() {
      this.startLoader();
      this.$store
        .dispatch('mytest/save')
        .catch(({ message }) => this.errorToast(message))
        .finally(() => this.endLoader());
    },
    onReset() {
      this.startLoader();
      this.$store
        .dispatch('mytest/reset')
        .catch(({ message }) => this.errorToast(message))
        .finally(() => this.endLoader());
    },
  },
};
</script>
```

Preview:

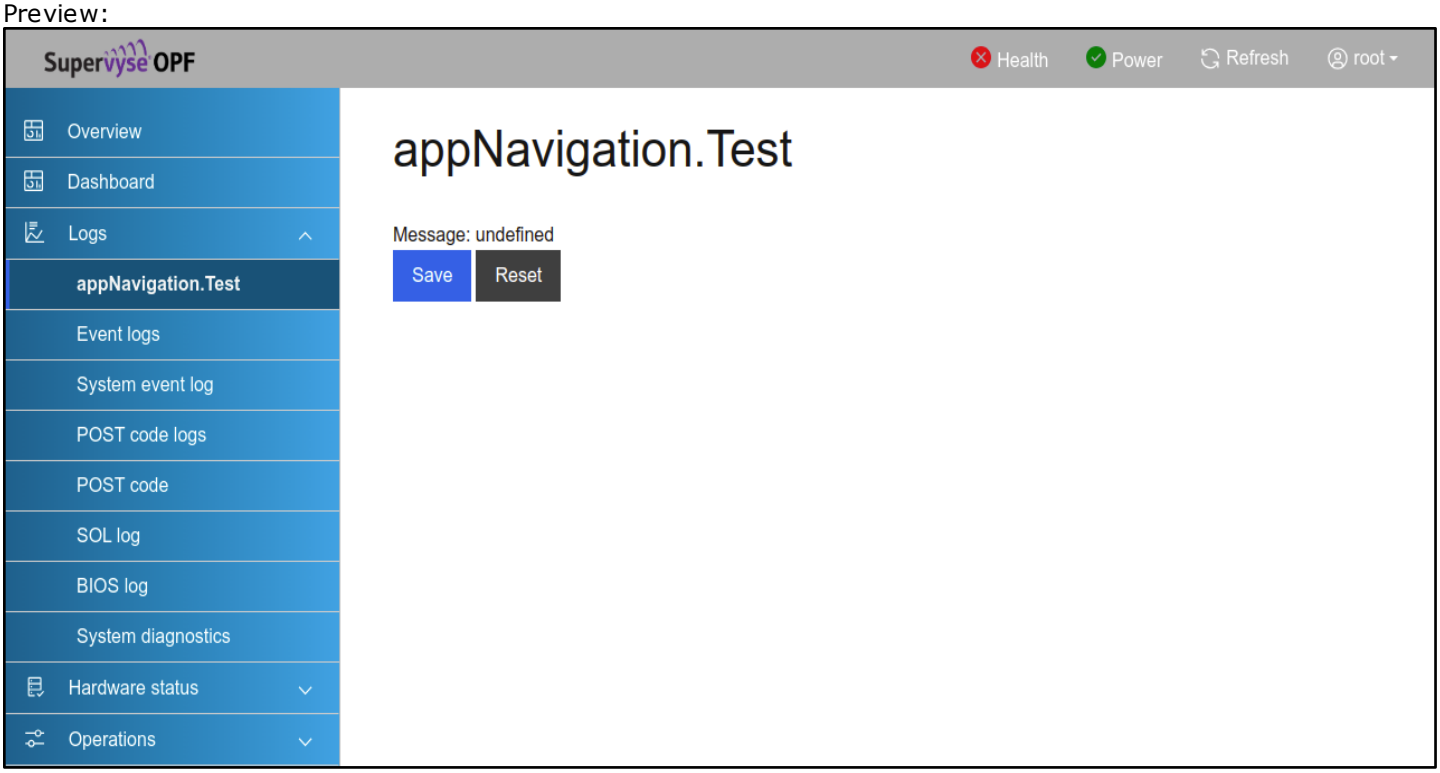

Figure 2.1.2-1 Preview

2.1.3 Add the Appropriate Language Strings

Add new language strings mapped to the language configuration files to support multiple languages by defining keys and their corresponding strings.

Table 2.1.3-1 Language settings

Location	Description
isc-src/web/webui-vue/src/env/insyde/locale/en-US.json	English language file.
isc-src/web/webui-vue/src/env/insyde/locale/zh-CN.json	Simplified Chinese language file.
isc-src/web/webui-vue/src/env/insyde/locale/zh-TW.json	Traditional Chinese language file.

Notice: If you want to add a new language, please refer to 3. Customize a New Language for detailed instructions.

The language configuration file is a JSON-formatted file composed of key-value pairs, allowing you to define your own custom strings. An example is shown below:

```
{
  "page1": {
    "action": {
      "add": "Add me",
      "cancel": "Cancel"
    },
    "key1": "value1",
    "key2": "value2",
    "toast": {
      "toast1": "toast_value1",
```

```
      "toast2": "toast_value2"
    }
  },
  "page2": {
    "key1": "value1",
    "key2": "value2"
  }
}
```

You can access the string using a format like this: `this.$t('page1.action.add')` to retrieve the string "Add me".

Example: isc-src/web/webui-vue/src/env/insyde/locale/en-US.json

```
diff --git a/web/webui-vue/src/env/insyde/locales/en-US.json b/web/webui-vue/src/env/insyde/locales/en-US.json
index 3358a02c4e..a6545eb22e 100644
--- a/web/webui-vue/src/env/insyde/locales/en-US.json
+++ b/web/webui-vue/src/env/insyde/locales/en-US.json
@@ -235,6 +235,7 @@
     "titleRefresh": "Refresh application data"
   },
   "appNavigation": {
+    "Test": "@:appPageTitle.Test",
     "snmp": "@:appPageTitle.snmp",
     "tacacs": "@:appPageTitle.tacacs",
     "virtualFrontPanel": "@:appPageTitle.virtualFrontPanel",
@@ -311,6 +312,7 @@
     "powerStatistics": "@:appPageTitle.powerStatistics"
   },
   "appPageTitle": {
+    "Test": "Test",
     "snmp": "SNMP",
     "tacacs": "TACACS+",
     "virtualFrontPanel": "Virtual front panel",
```

Figure 2.1.3-1 Diff tool comparison of en-US.json

Notice: "@:appPageTitle.Test" serves as a reference to "appPageTitle.Test".

Preview:

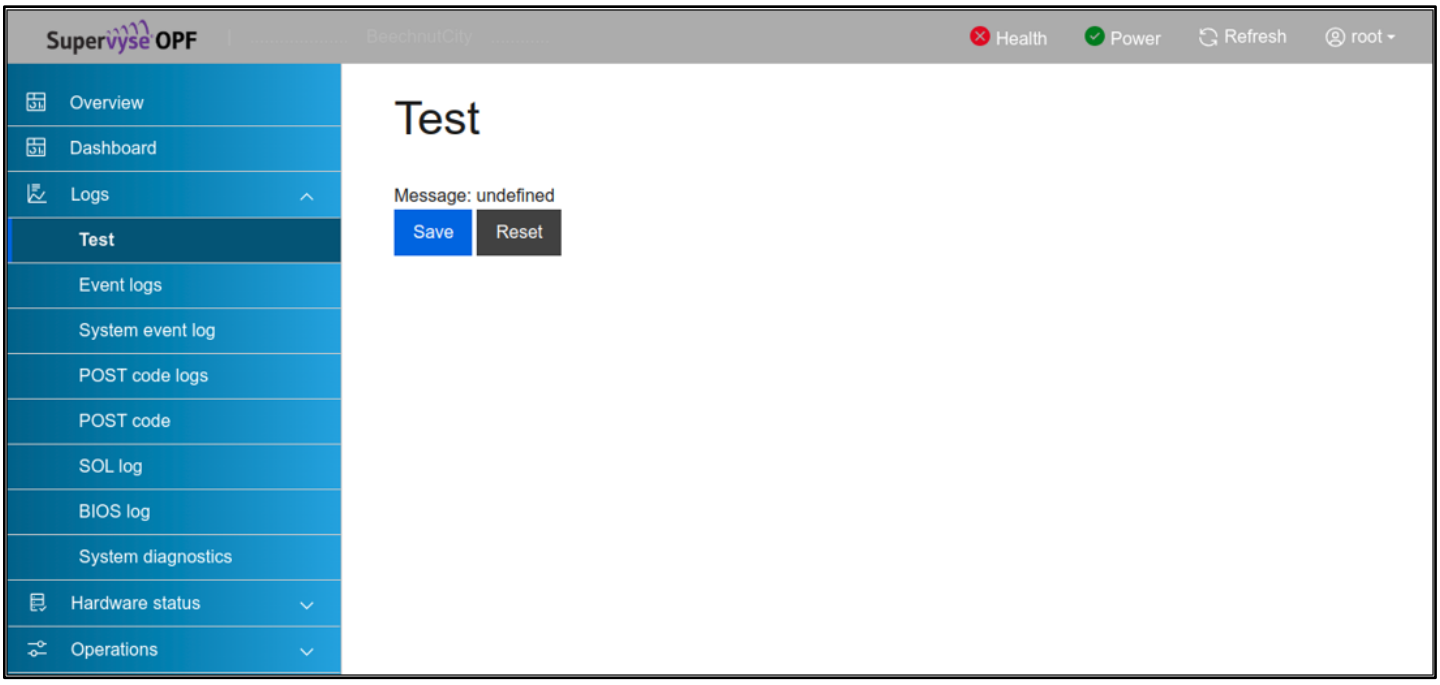

Figure 2.1.3-2 Preview

2.1.4 Add Necessary Stores for State Management

In the Vue framework, particularly when using Vuex (the state management library for Vue), a "store" is a centralized repository for managing the state of your application.

Add a new store<TestStore> and named it mytest.
Location: isc-src/web/webui-vue/src/store/index.js

```
diff --git a/web/webui-vue/src/store/index.js b/web/webui-vue/src/store/index.js
index d7c1b22d03..adbb82ae7d 100644
--- a/web/webui-vue/src/store/index.js
+++ b/web/webui-vue/src/store/index.js
@@ -25,6 +25,7 @@ import BmcStore from './modules/HardwareStatus/BmcStore';
 import ProcessorStore from './modules/HardwareStatus/ProcessorStore';
 import AssemblyStore from './modules/HardwareStatus/AssemblyStore';
 import PostCodeLogsStore from './modules/Logs/PostCodeLogsStore';
+import TestStore from './modules/Logs/TestStore';
 import PoliciesStore from './modules/SecurityAndAccess/PoliciesStore';
 import FactoryResetStore from './modules/Operations/FactoryResetStore';

@@ -64,6 +65,7 @@ export default new Vuex.Store({
     processors: ProcessorStore,
     assemblies: AssemblyStore,
     postCodeLogs: PostCodeLogsStore,
+    mytest: TestStore,
     virtualMedia: VirtualMediaStore,
     policies: PoliciesStore,
     factoryReset: FactoryResetStore,
```

Figure 2.1.4-1 Diff tool comparison of store

Location: isc-src/web/webui-vue/src/store/modules/Logs/TestStore.js

```
import api from '@/store/api';

const TestStore = {
  namespaced: true,
  state: {
    message: 'N/A',
  },
  getters: {
    message: (state) => state.message,
  },
  mutations: {
    setMessage: (state, msg) => {
      state.message = msg;
    },
  },
  actions: {
    async save({ commit }) {
      return await api
        .get('/cgi/mypage.cgi')
        .then(({ data = {} }) => commit('setMessage', data.POWER_STATUS))
        .catch((error) => {
          console.log(error);
          throw new Error('Save error', error);
        });
    },
    async reset({ commit }) {
      return await api
        .get('/redfish/v1/Systems/system')
        .then(({ data = {} }) => commit('setMessage', data.Model))
        .catch((error) => {
          console.log(error);
          throw new Error('Reset error', error);
        });
    },
  },
};

export default TestStore;
```

Preview:

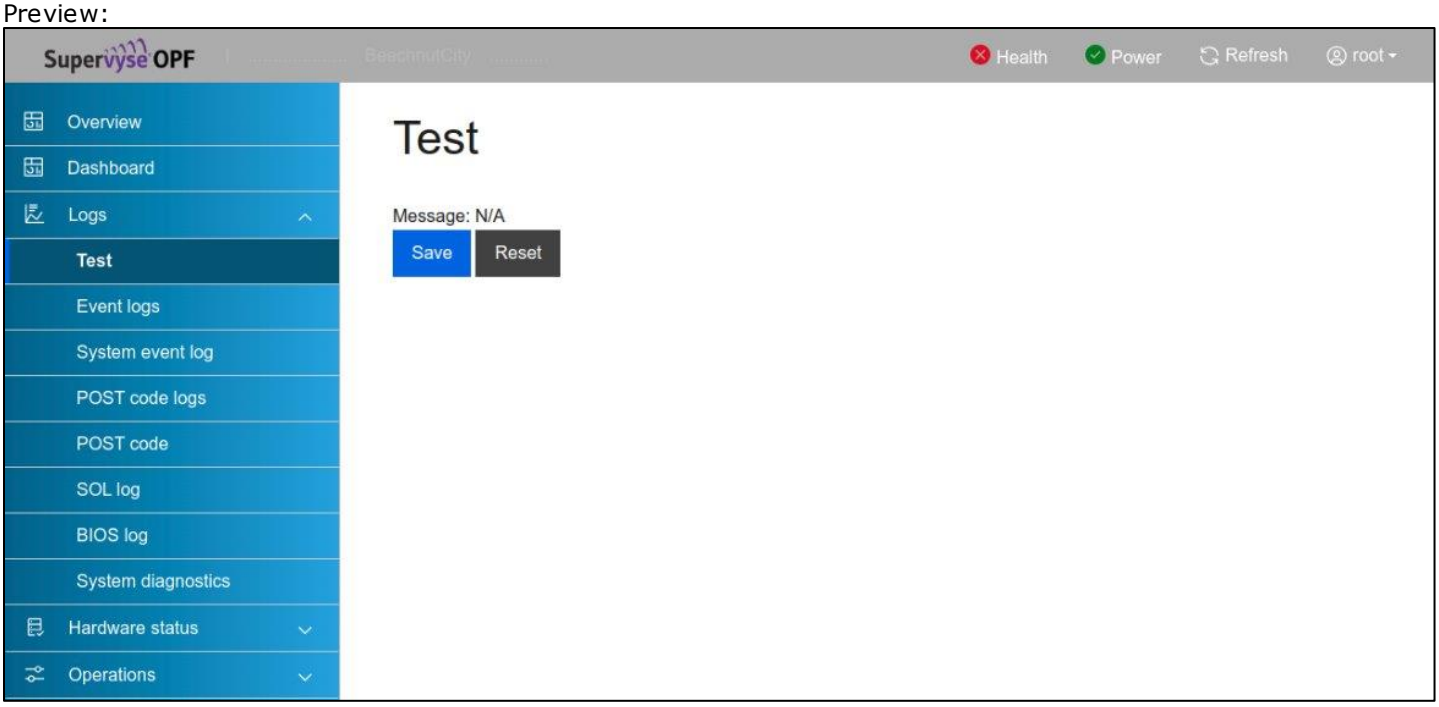

Figure 2.1.4-2 Preview

2.2 Backend

A Backend Web API (Application Programming Interface) is a server-side application or set of services that handle requests from clients, typically web or mobile applications. Web APIs provide data or functionality that can be accessed over HTTP(S) through endpoints (URLs), allowing various front-end interfaces (such as Vue applications) to interact with the backend for data or actions.

The following sections will teach you how to add a new CGI Web API.

2.2.1 CGI Source Code

The following example will add a new Web API to retrieve the current power status.

URL: https://xxx.xxx.xxx.xxx/cgi/mypage.cgi
Method: GET
Expected result:

```
{
  "POWER_STATUS": "off"
}
```

Location: isc-src/web/cgi/mypage.c

```
#include <utils.h>
#include <mci_chassis.h>

static void get_data(cgi_params *cpsp)
{
```

```
    int ret = 0;
    int status = 0;
    int data_length = 0;

    ret = mci_get_chassis_status(&status, NULL, &data_length);
    if (ret == 0) {
        cJSON *json = cJSON_CreateObject();
        if (json) {
            cJSON_AddStringToObject(json, "POWER_STATUS", status == 1 ? "on" : "off");
            exit_json(cpsp, json);
        }
        exit_error(cpsp, 500, "cJSON_CreateObject fail");
    }
    exit_error(cpsp, 500, "mci_get_chassis_status fail");
}

int main(void)
{
    HANDLER handler_table[] = {
        /* method, query, privilege, handler_function */
        { "GET",  NULL, 2, get_data },
    };

    PROCESS_REQUEST(handler_table);
    return 0;
}
```

2.2.2 Build

```
# Perform a module clean for CGI.
bitbake cgi -c clean

# Perform a module compile for CGI.
bitbake cgi
```

After the module is built, libcgiapi.so will be updated at the location isc-src/web/cgi/oe-workdir/package/usr/lib/libcgiapi.so

2.2.3 Deploy

Replace libcgiapi.so on the BMC console and restart the bmcweb service to reload the .so file.

```
# <On the BMC console>
# Copy the new libcgiapi.so to BMC.
scp user@xxx.xxx.xxx.xxx:/yourSDK/isc-src/web/cgi/oe-workdir/package/usr/lib/libcgiapi.so /usr/lib/

# Restart bmcweb service.
systemctl restart bmcweb
```

2.3 Verify

This section guides you on how to validate your modifications.

Navigate to the Test page.

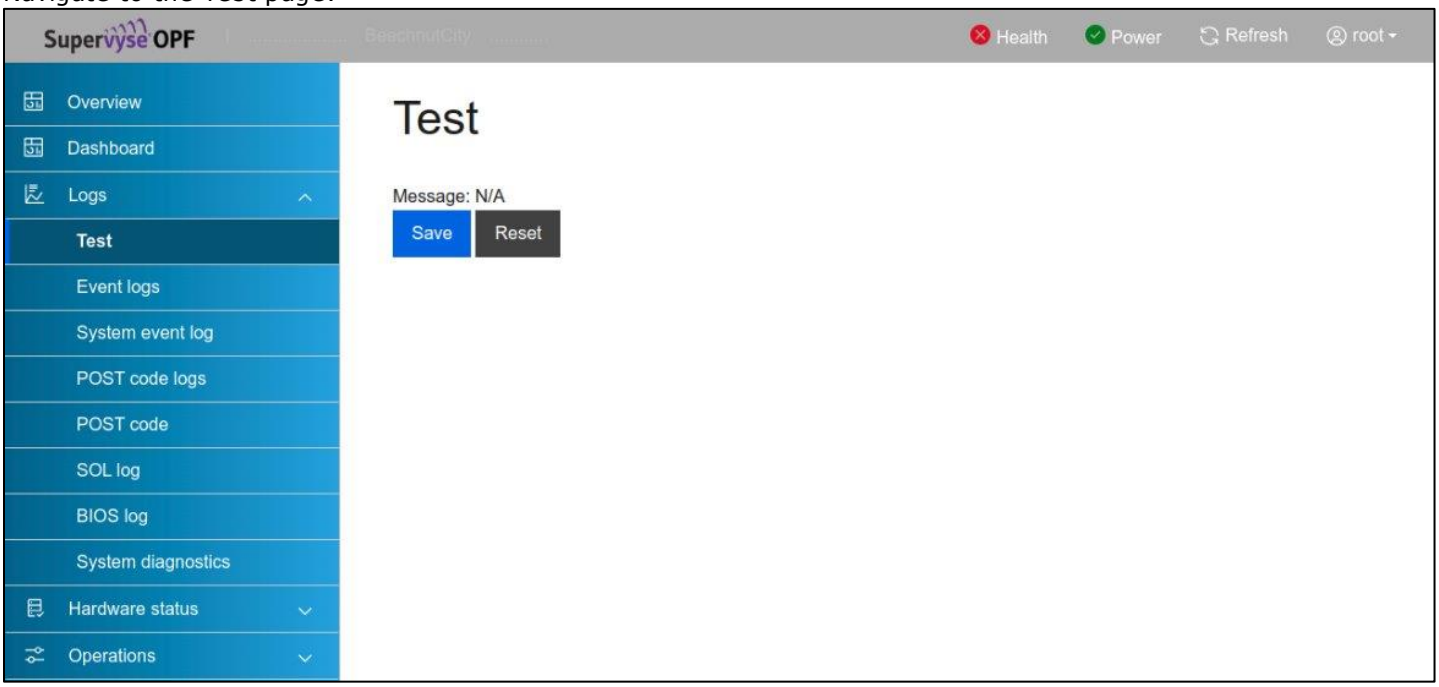

Figure 2.3-1 Verify - part 1

Press F12 on your keyboard to open the browser's "Developer Tools".

Click the "Reset" button and review the response.

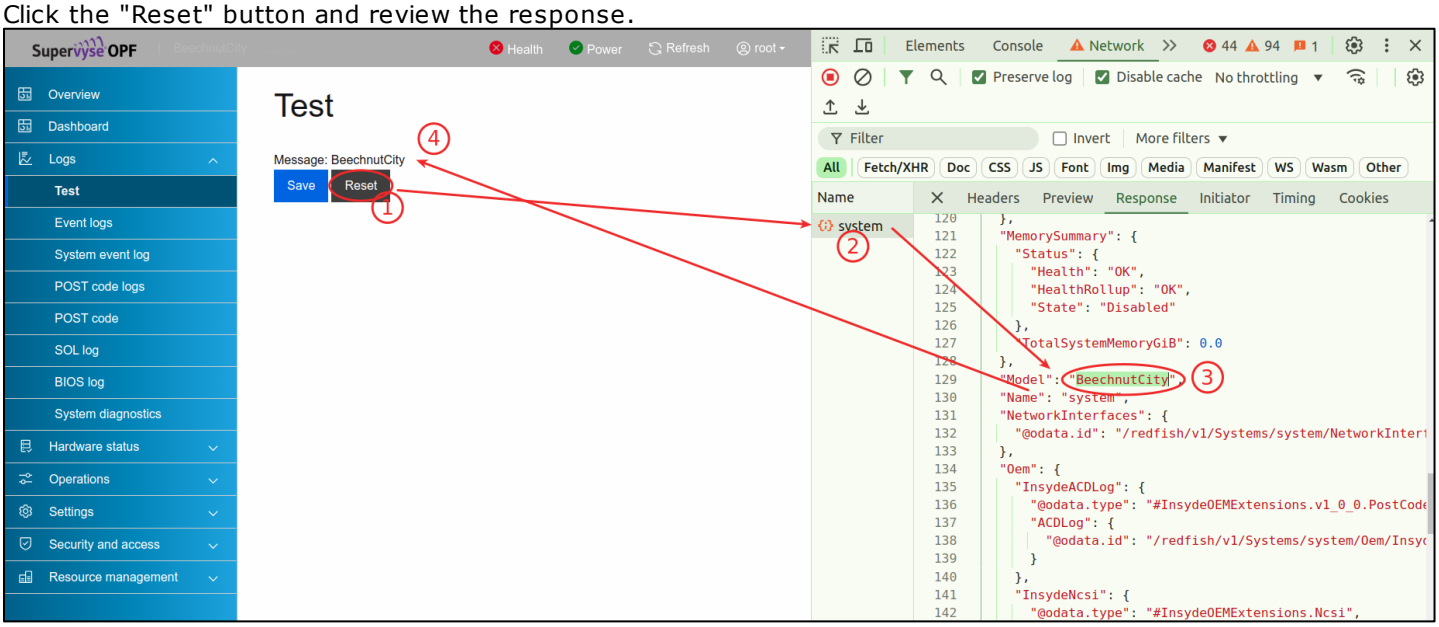

Figure 2.3-2 Verify - part 2

Click the "Save" button and review the response.

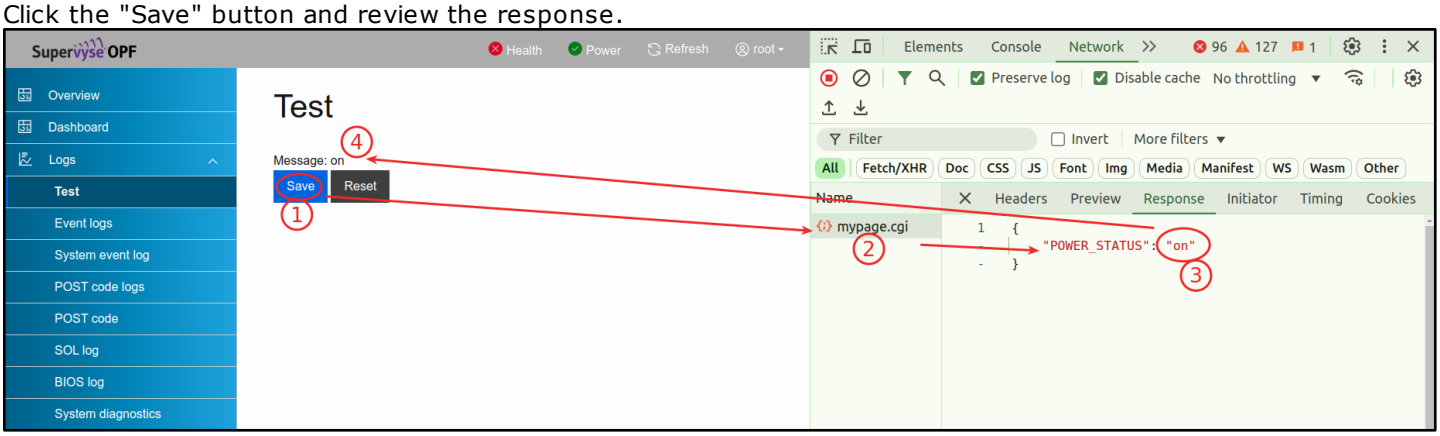

Figure 2.3-3 Verify - part 3

2.4 Customize Page Visibility

The Page visibility can be managed through *menuconfig*. This setting only controls visibility; proper functionality depends on platform compatibility and underlying support.

Steps:

1. Run the command: bitbake pkconfig -c menuconfig
2. Select the corresponding pages to enable the full-page function.

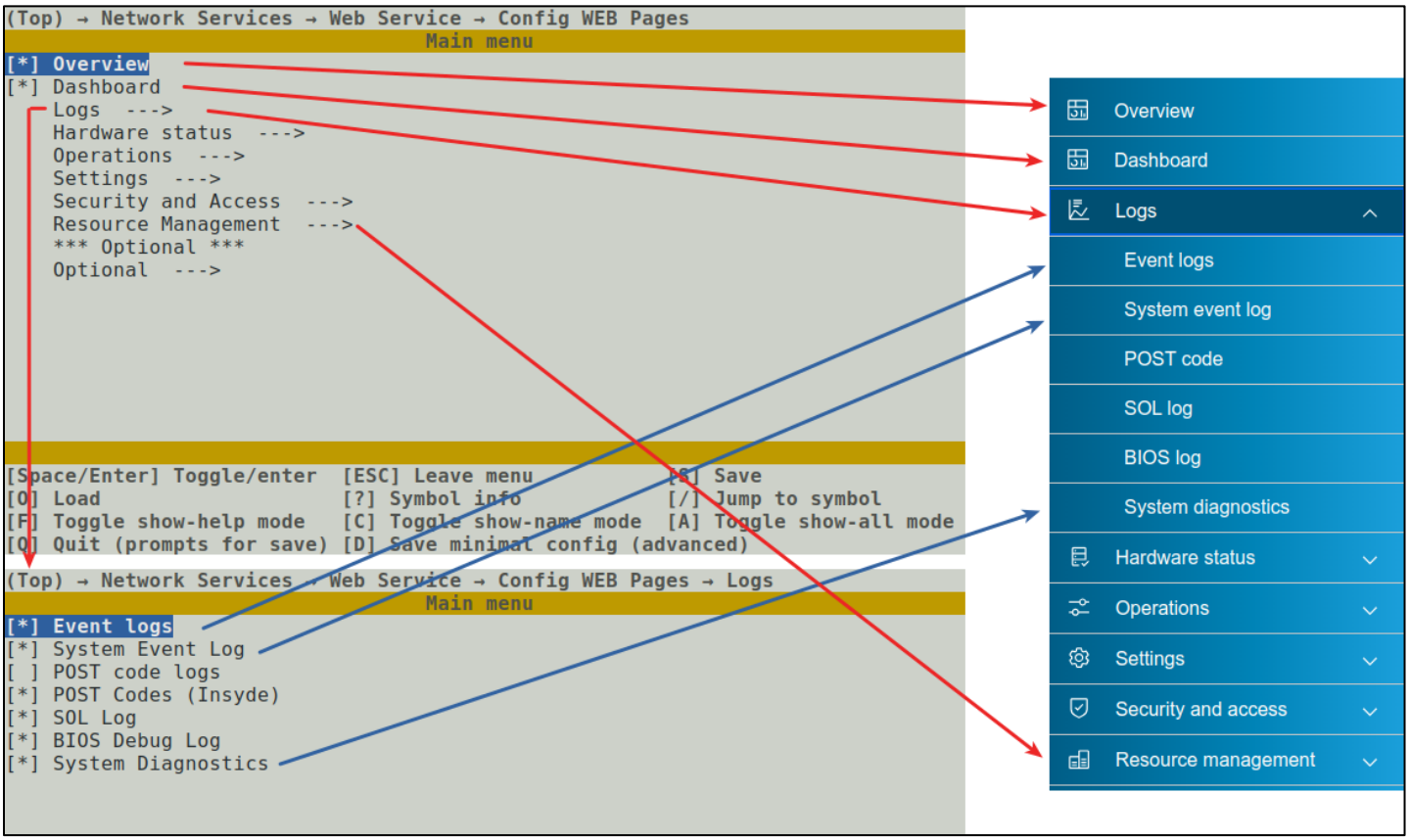

Figure 2.4-1 Page menuconfig

Another way to customize page visibility is by modifying the menu.json file, such as **isc-src/web/webui-vue/src/env/insyde/json/menu.json**, or the menu.json file in the **isc-src/web/optional/...** directory.

You can use the "**disable**" property to customize the page visibility.

```
1  {
2      "overview":
3      {
4          "index": 1,
5          "label": "appNavigation.overview",
6          "route": "/",
7          "component": "@/views/Overview",
8          "view": [2, 3, 4],
9          "change": [3, 4],
10         "icon": "iconOverview",
11         "disable": true,
12         "depends": [
13             "CONFIG_OFD_PAGE_OVERVIEW"
14         ]
15
16     },
```

Figure 2.4-2 Page disable

3. Customize a New Language

This section provides guidance on adding and configuring a new language for your web pages. Below is an example of how to add it.

Add a new language option to the login page with the following details:

- **Language Name**: New-English
- **Value**: en-NEW

Location: isc-src/web/webui-vue/src/views/Login/Login.vue

```
164        languages: [
165          {
166            value: 'en-US',
167            text: 'English',
168          },
169          {
170            value: 'zh-CN',
171            text: '简体中文',
172          },
173          {
174            value: 'zh-TW',
175            text: '繁體中文',
176          },
177          {
178            value: 'en-NEW',
179            text: 'New-English',
180          },
181          // Disable langauages es & ru in temporary(during the Supervyse vue pages development).
182          //{
183          //  value: 'es',
184          //  text: 'Español',
185          //},
186          //{
187          //  value: 'ru-RU',
188          //  text: 'Русский',
189          //},
190        ],
```

Figure 3-1 Detail of Login.vue

Add the following two language configuration files:

- **Location 1**: isc-src/web/webui-vue/src/locales/en-NEW.json - The native language file.

```
{
  "pageLogin": {
    "label_please_login": "Please Login",
    "username": "Username",
    "logIn": "Click to log in to the BMC"
  }
}
```

- **Location 2**: isc-src/web/webui-vue/src/env/insyde/locales/en-NEW.json - The Insyde override language file.

```
{
  "pageLogin": {
    "label_please_login": "Please Login :)",
```

```
    "username": "Username"
  }
}
```

Result:

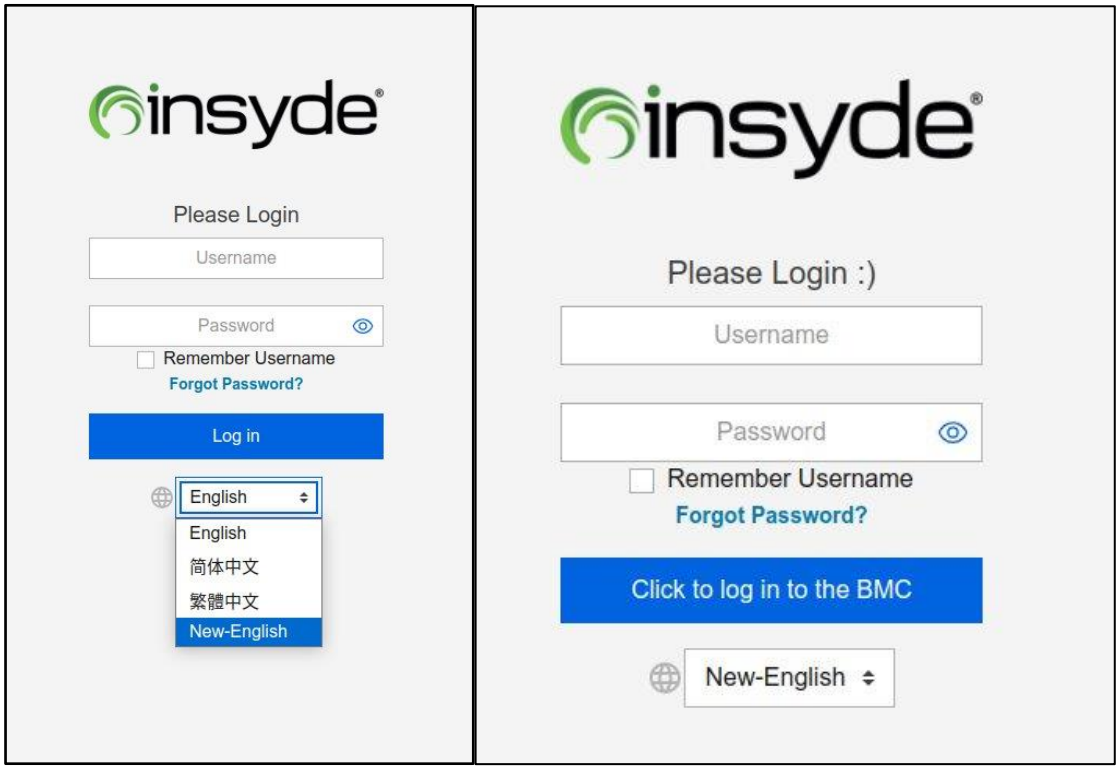

Figure 3-2 Review of adding a new language

3.1 Language Key Fallback Policy

The Language Key Fallback Policy ensures that untranslated or missing language keys do not disrupt the user experience by providing a structured fallback mechanism. Key aspects of the policy include:

- **Primary Language Selection**: The system retrieves text from the user's selected language pack (e.g., Chinese or Japanese).
- **Fallback to Default Language**: If the key is missing in the selected language pack, it falls back to the default language pack, which is en-US.
- **Override File Priority**: Within the same language pack, the override file is checked first for the key.
- **Fallback to Native File**: If the key is not found in the override file, the native file is used.

This process ensures seamless usability by preventing broken or empty text displays, providing a consistent experience across all languages.

You can change the fallback default language pack by editing the file: isc-src/web/webui-vue/src/i18n.js.

```
43  export default new VueI18n({
44    // Get default locale from local storage
45    locale: window.localStorage.getItem('storedLanguage'),
46    // Locales that don't exist will fallback to English
47    fallbackLocale: 'en-US'
48    // Falling back to fallbackLocale generates two console warnings
49    // Silent fallback suppresses console warnings when using fallback
50    silentFallbackWarn: true,
51    messages: loadLocaleMessages(),
52  });
```

Figure 3.1-1 Change the fallback default language pack

Example: The key "logIn" cannot be found. Trying to find it in the native language file.

Notice: String key searches follow the order below:

1. isc-src/web/webui-vue/src/env/insyde/locales/en-NEW.json
2. isc-src/web/webui-vue/src/locales/en-NEW.json
3. Fallback language defaults to en-US.json
4. No translation.

- Location: isc-src/web/webui-vue/src/locales/en-NEW.json

```
{
  "pageLogin": {
    "label_please_login": "Please Login",
    "username": "Username",
    "logIn": "Click to log in to the BMC"
  }
}
```

Attempting to locate it in the native language file.

- Location: isc-src/web/webui-vue/src/env/insyde/locales/en-NEW.json

```
{
  "pageLogin": {
    "label_please_login": "Please Login :)",
    "username": "Username"
  }
}
```

Cannot find key "logIn".

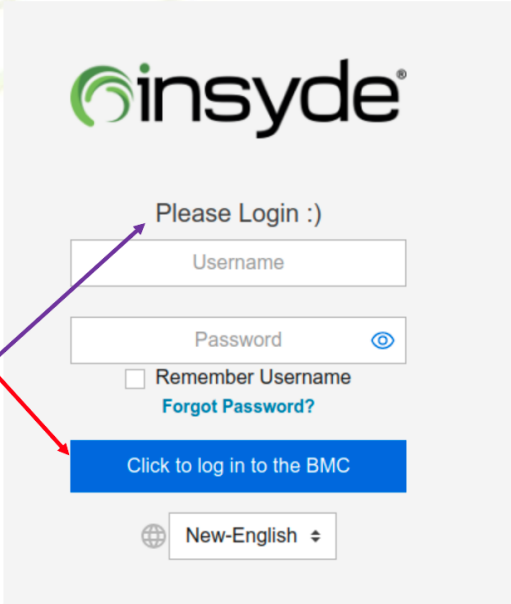

Figure 3.1-1 Language Key Fallback to Native Language

4. Customize the Web Logo

This section offers a step-by-step guide on customizing the page logo.

Resource location: isc-src/web/webui-vue/src

Figure 4-1 Resource images

Screen preview with resource files mapping.

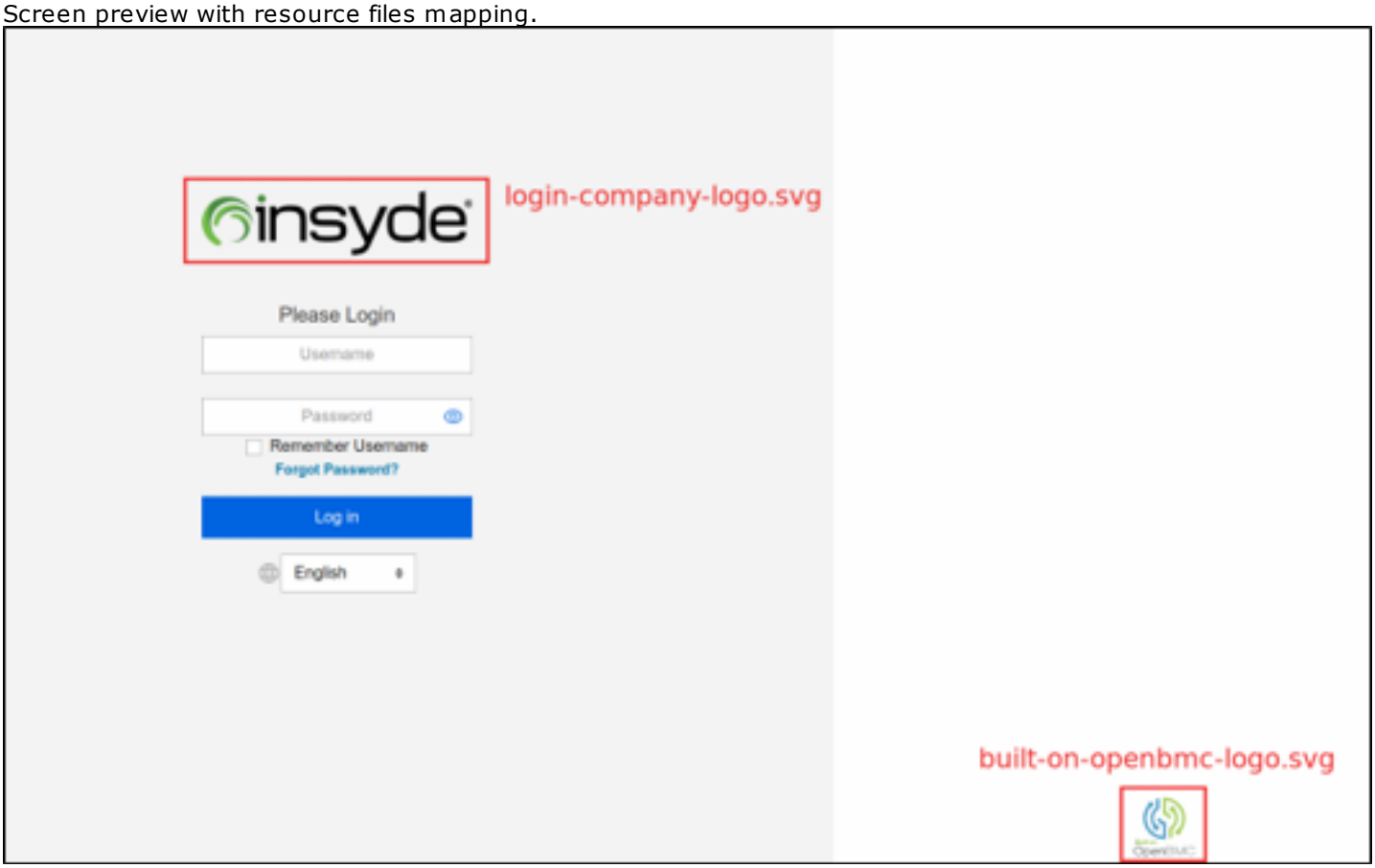

Figure 4-2 Preview and resource files mapping - part 1

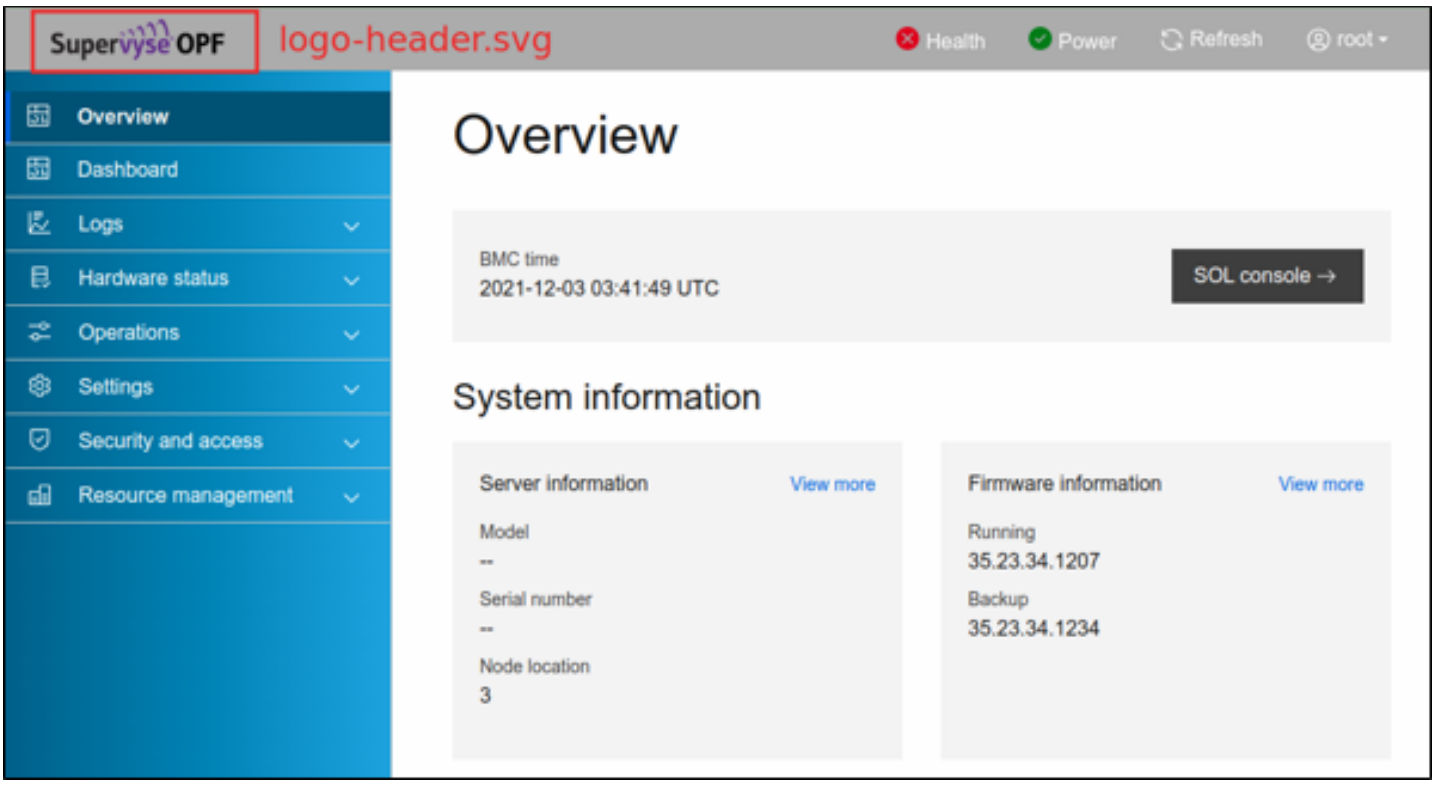

Figure 4-3 Preview and resource files mapping - part 2

4.1 Replace the Logos

Update the logo and include additional images in the resource folder.

- Location: isc-src/web/webui-vue/src/assets/images
- Add a new image file named logo-side.svg.
- Replace login-company-logo.svg with your company's logo.
- Replace logo-header.svg with your company's logo as well.

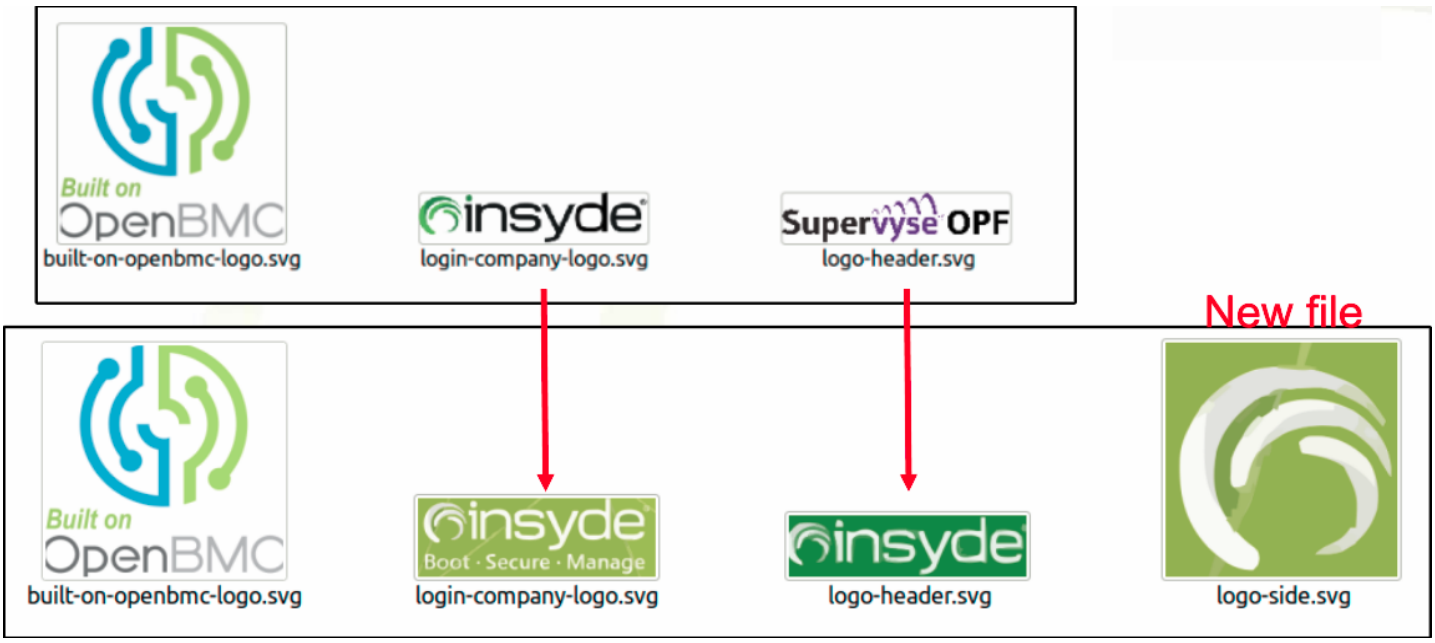

Figure 4.1-1 Replace the logos

Modify the Vue file for the login page to update the logo.

- Location: isc-src/web/webui-vue/src/layouts/LoginLayout.vue
- Add an icon to the left side of the OpenBMC logo.

```
13          <h1 v-if="customizableGuiName" class="h3 mb-5">
14            {{ customizableGuiName }}
15          </h1>
16          <router-view class="login=form form-background" />
17        </div>
18      </div>
19      <div class="login-aside">
20        <div class="login-aside__logo-brand">
21          <!-- Add Secondary brand logo if needed -->
22          <img
23            height="60px"
24            src="@/assets/images/logo-side.svg"
25            alt="Built on OpenBMC"
26          />
27        </div>
28        <div class="login-aside__logo-bmc">
29          <img
30            height="60px"
31            src="@/assets/images/built-on-openbmc-logo.svg"
32            alt="Built on OpenBMC"
33          />
34        </div>
```

Figure 4.1-2 Add the logo to the side

The comparison images after the modifications are shown below.
Comparison: Login Page (Before vs. After)

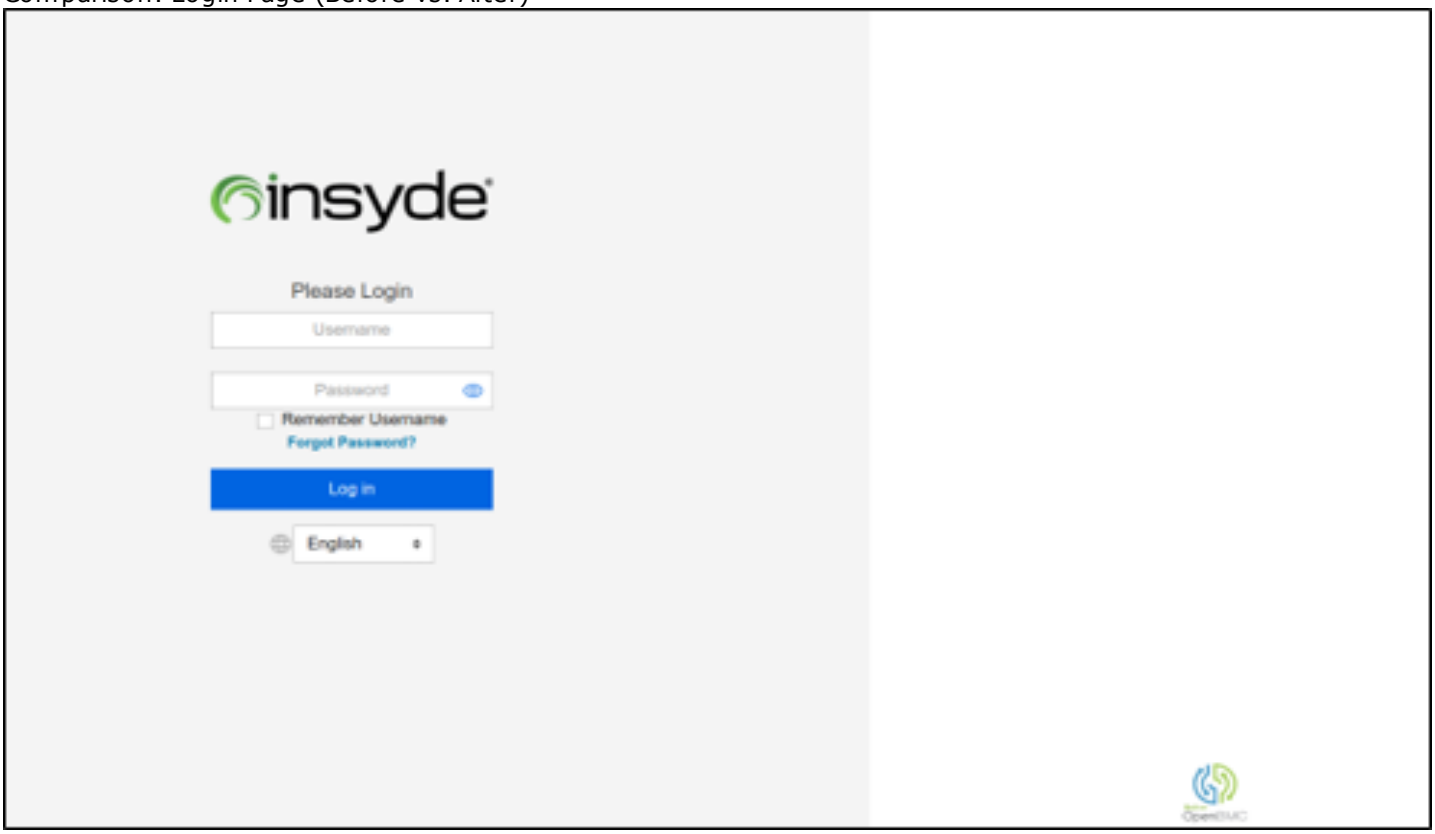

Figure 4.1-3 Login Page - Before

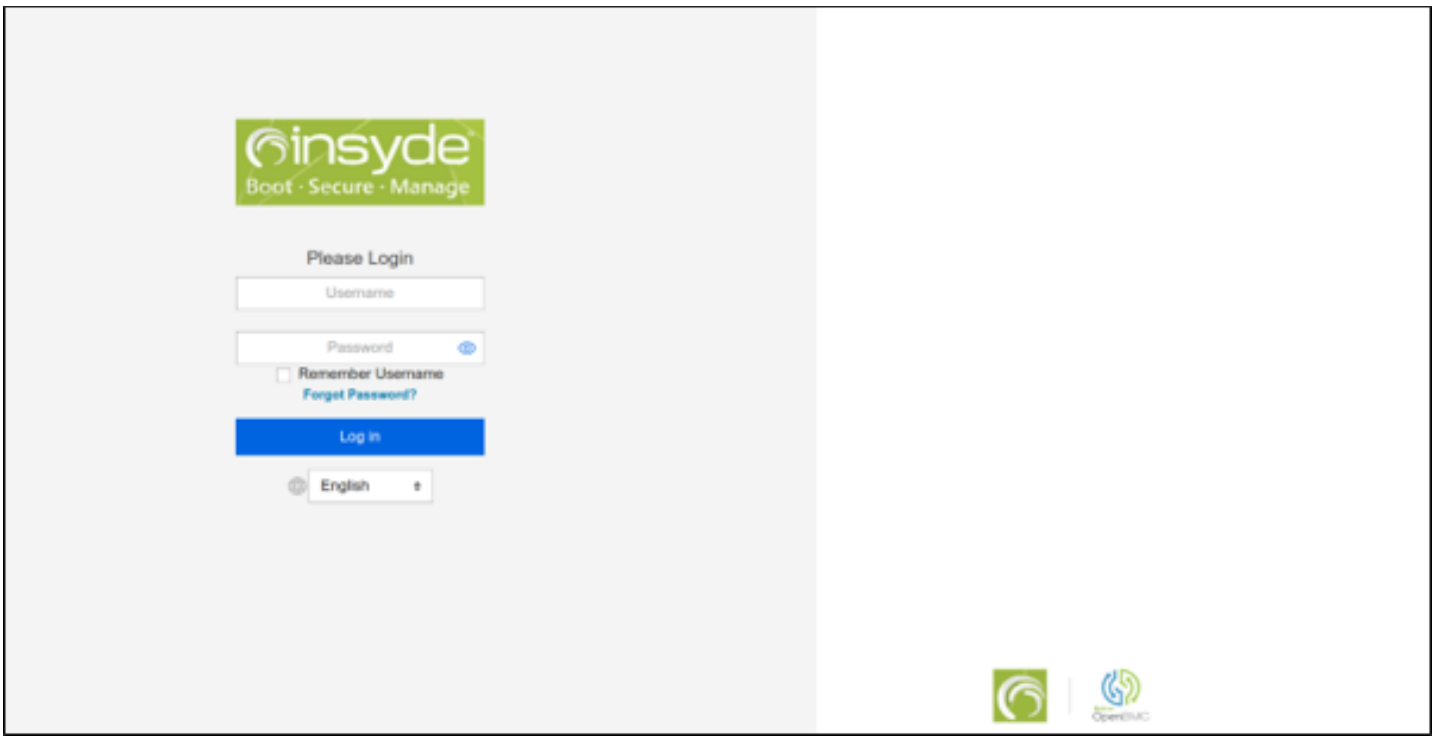

Figure 4.1-4 Login Page - After

Comparison: Normal Page (Before vs. After)

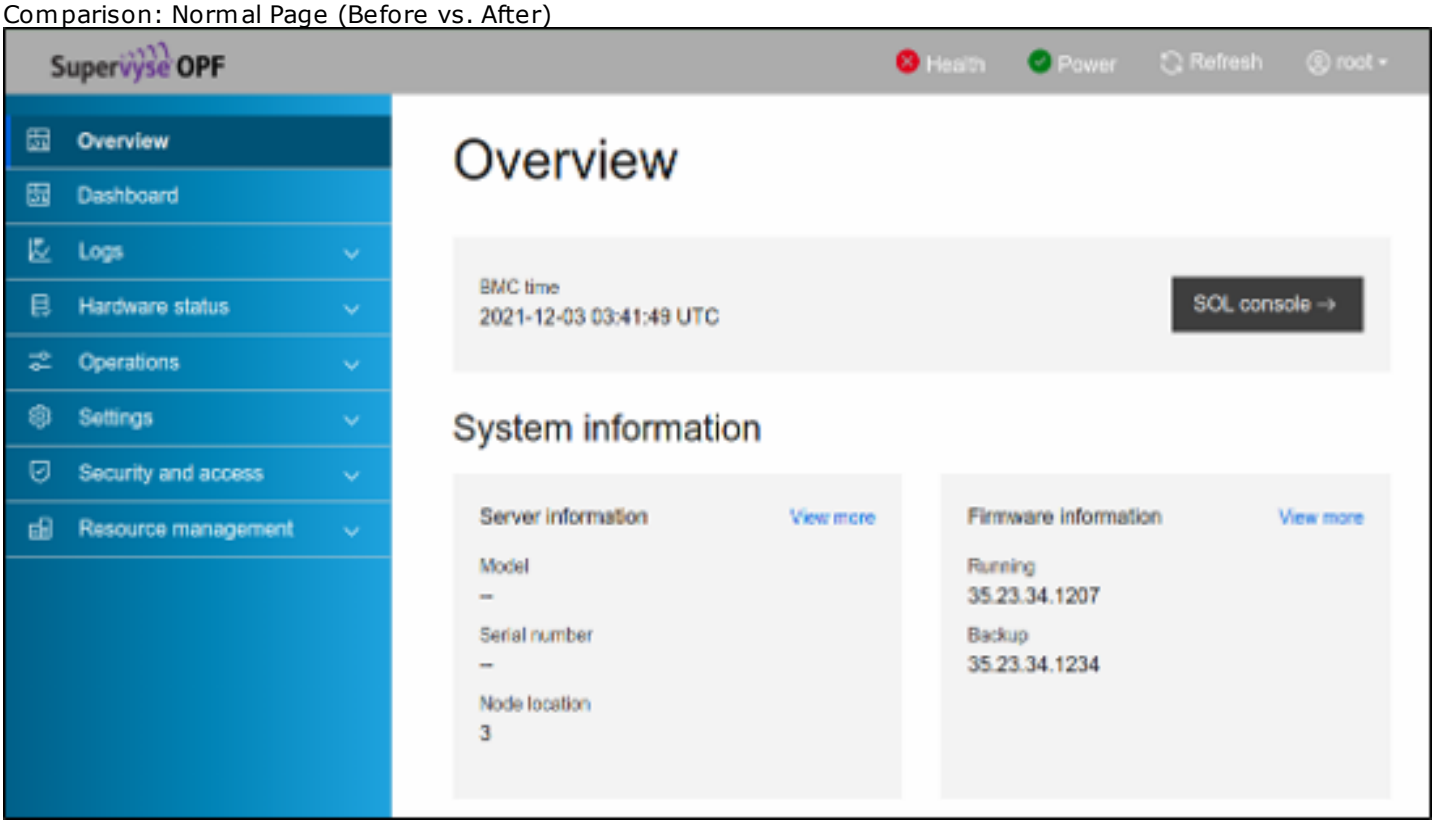

Figure 4.1-5 Normal Page - Before

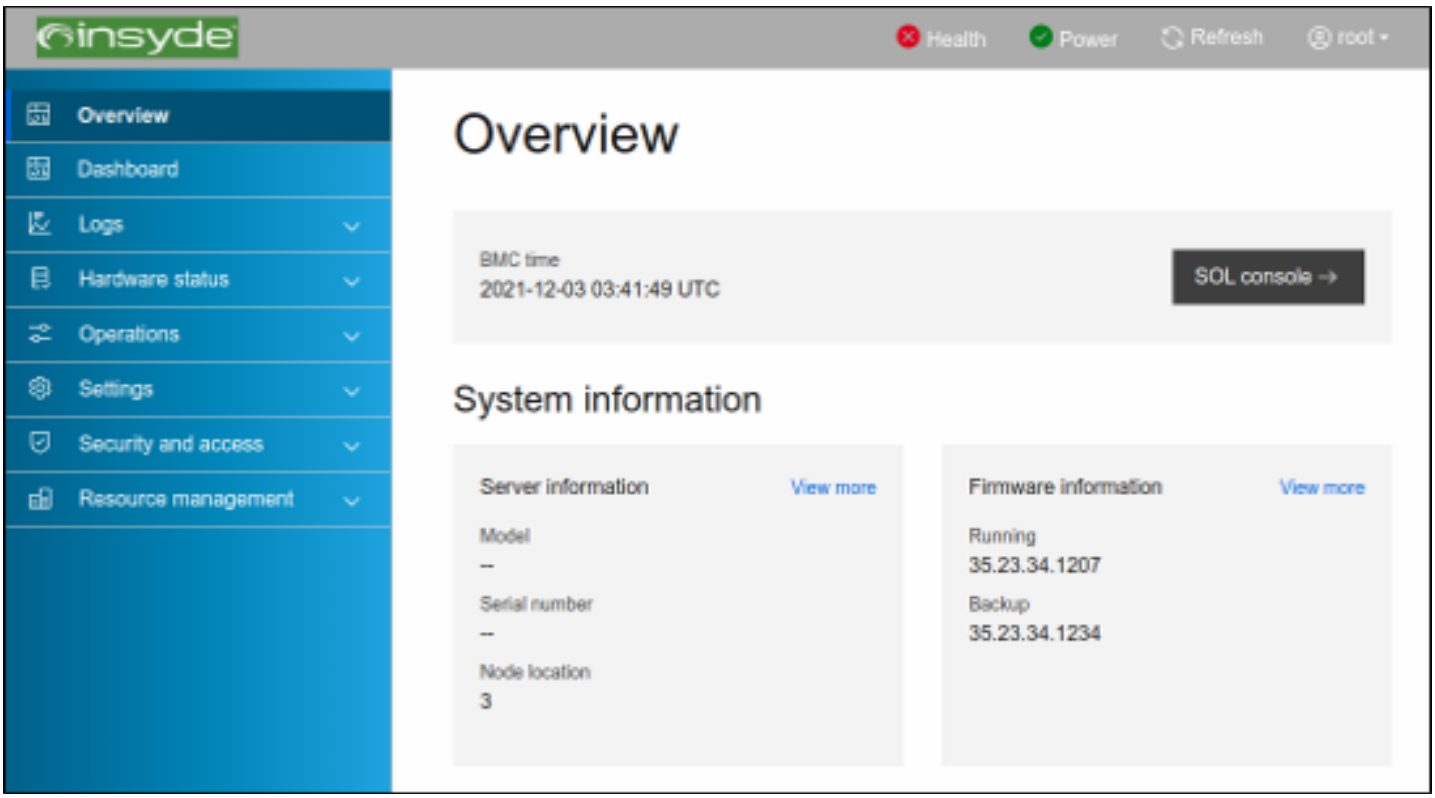

Figure 4.1-6 Normal Page - After

4.2 Reference

The customization guide for the logos on web pages is based on the OpenBMC guidance provided below:
1. https://github.com/openbmc/docs/blob/master/development/web-ui.md
2. https://github.com/openbmc/webui-vue/issues/63

The first page describes how to replace the login-company-logo image on the login page and the logo-header image on every page after login.

The second page describes how to put your company logo alongside the built-on-openbmc-logo image on the bottom right corner of the login page. It also suggests that the built-on-openbmc-logo image is not replaceable.

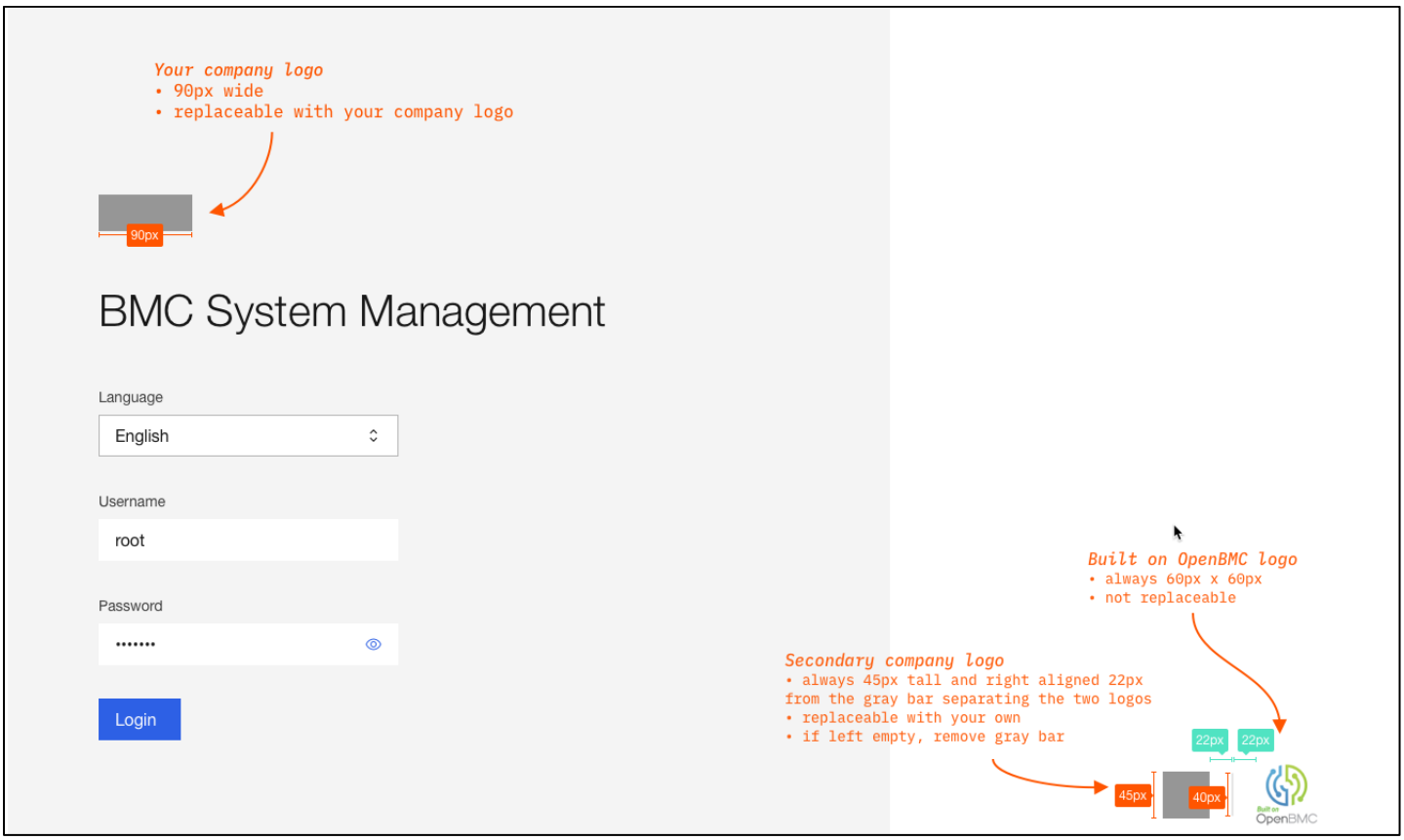

Figure 4.2-1 How to put your company logo - part 1

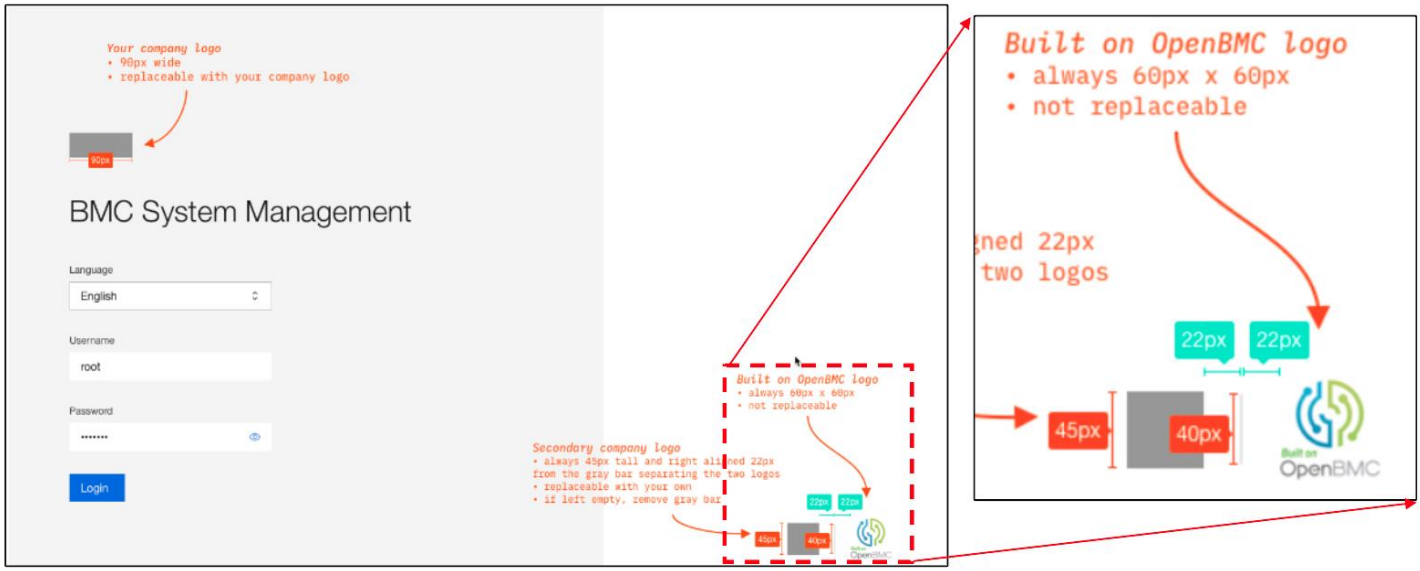

Figure 4.2-2 How to put your company logo - part 2

OpenBMC is one of the trademarks owned by LF Projects. For further information about their policy, you may like to look at their Trademark Policy page: https://lfprojects.org/policies/trademark-policy/

5. Customize the Menu Style

This section explains how to customize the menu style. As shown in the image below, you can customize your menu's appearance.

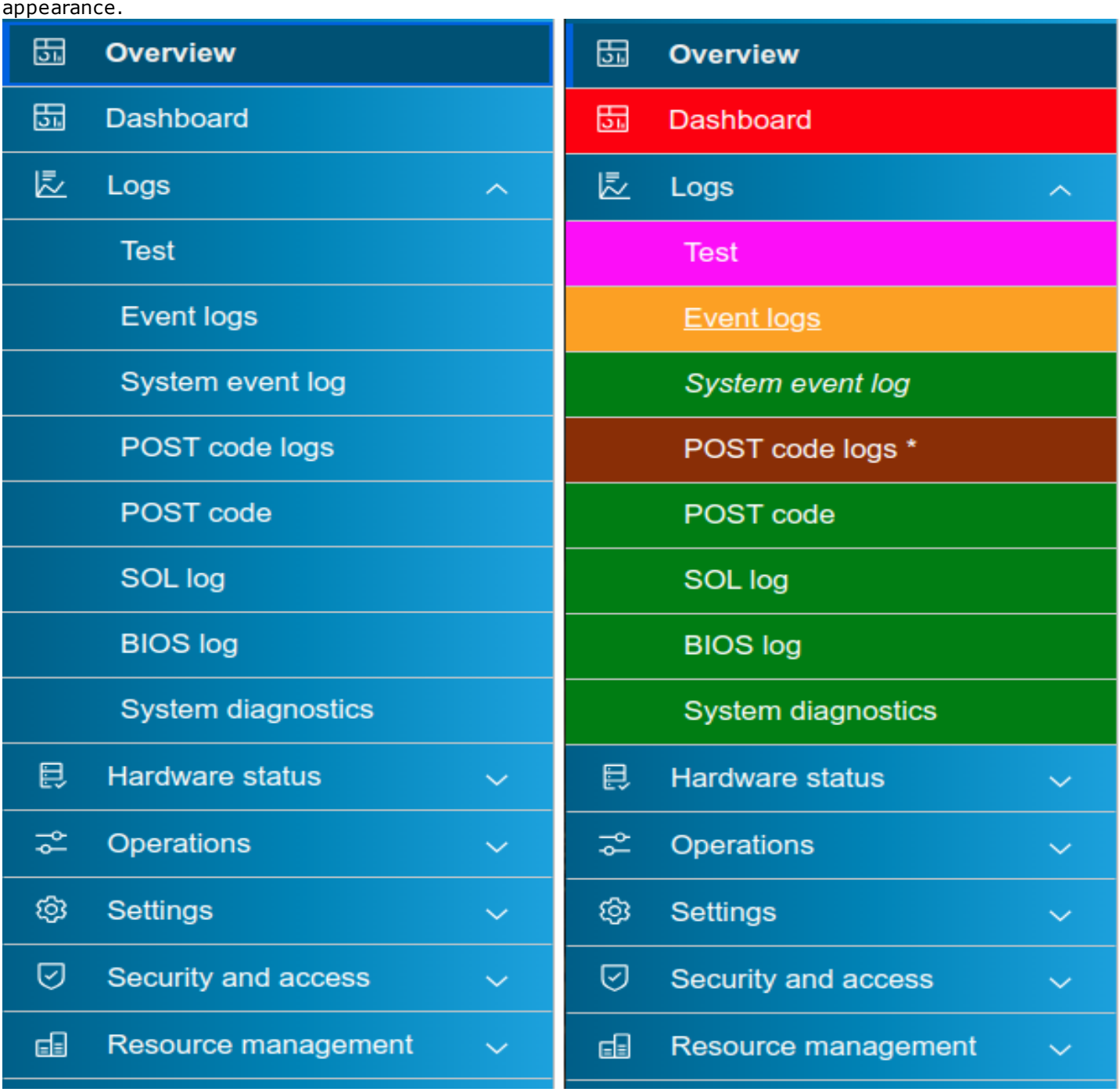

Figure 5-1 Menu styles

5.1 Enable Menu Style

First, enable Demo mode, as the menu style is only functional in this mode.

Location: isc-src/web/webui-vue/.env.insyde
Add the following setting to enable it.

```
VUE_APP_DEMO=true
```

=true (default on)
=false or SKIP (default off)
The default value of demo is stored in Session Storage.

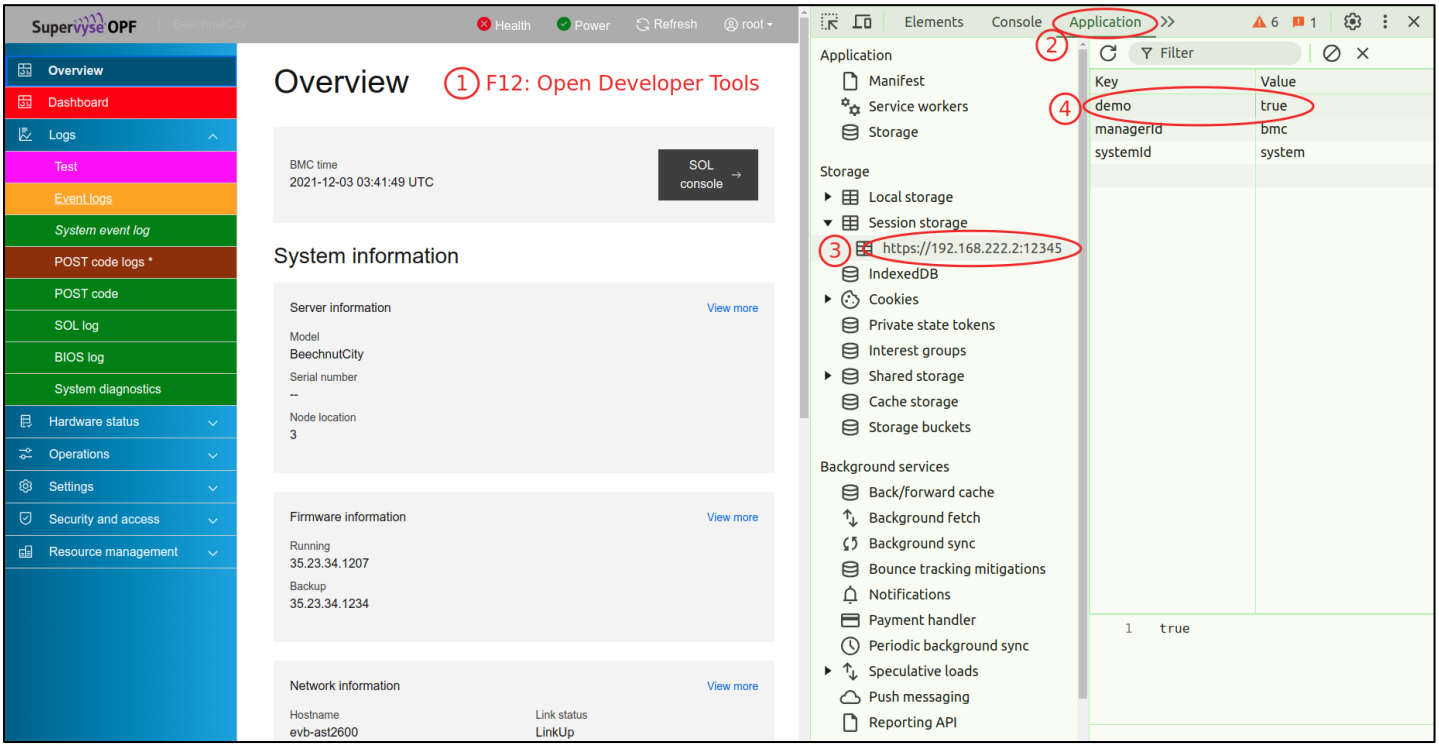

Figure 5.1-1 Web developer tools

1. Press F12 on your keyboard to open the browser's "Developer Tools".
2. Select the "Application" tab.
3. Choose "Session storage" from the left-hand menu.
4. Variables are listed in the key/value format on the right side.
   Toggle the demo variable (true or false) and press F5 (browser reload) to apply.
   This value will reset to default when the page is closed and reopened.

5.2 Configure Menu Style

This section provides guidance on how to customize the style of the menu.

Location: isc-src/web/webui-vue/src/env/insyde/json/menu.json
Add or modify the "myclass" for a menu item to customize its style.

```
diff --git a/web/webui-vue/src/env/insyde/json/menu.json b/web/webui-vue/src/env/insyde/json/menu.json
index 64c6d548f8..96348e3422 100644
--- a/web/webui-vue/src/env/insyde/json/menu.json
+++ b/web/webui-vue/src/env/insyde/json/menu.json
@@ -14,7 +14,7 @@
     "dashboard":
     {
         "index": 2,
-        "myclass": "green",
+        "myclass": "red",
         "label": "appNavigation.dashboard",
         "route": "/dashboard",
         "component": "@/env/insyde/views/Dashboard",
@@ -35,6 +35,7 @@
             "test-logs":
             {
                 "index": 1,
+                "myclass": "rgb(255,0,255)",
                 "label": "appNavigation.Test",
                 "route": "/logs/test",
                 "component": "@/views/Logs/Test",
@@ -45,6 +46,7 @@
             "event-logs":
             {
                 "index": 1,
+                "myclass": "orange underline",
                 "label": "appNavigation.eventLogs",
                 "route": "/logs/event-logs",
                 "component": "@/views/Logs/EventLogs",
@@ -55,7 +57,7 @@
             "sel":
             {
                 "index": 2,
-                "myclass": "green",
+                "myclass": "green italic",
                 "label": "appNavigation.sel",
                 "route": "/logs/sel",
                 "component": "@/env/insyde/views/SEL",
@@ -68,6 +70,7 @@
             "post-code-logs":
             {
                 "index": 3,
+                "myclass": "#883300 insyde",
                 "label": "appNavigation.postCodeLogs",
                 "route": "/logs/post-code-logs",
                 "component": "@/views/Logs/PostCodeLogs",
```

Figure 5.2-1 Configure menu style

You can use the "myclass" property to customize the appearance of a menu item. Here's an example:

```
"myclass": "insyde italic bold underline color"
```

- **insyde**: Add the "insyde" class to include additional content (*) and apply the Insyde-selected back-groupd color.
- **italic**: Font style: italic.
- **bold**: Font style: bold. (Selected items are bold by default and are not prominent when using this style.)
- **underline**: Font style: underline.
- ***color***: Supported formats: *red* (color_name), *#ff0000*, or rgb(255, 0, 0).

User-defined insyde class.

- Location: isc-src/web/webui-vue/src/components/AppNavigation/AppNavigation.vue
- When you define "insyde" in myclass, a * will be appended at the end.
- You can change this appended value here.

```
182  .insyde a::after,
183  a.insyde::after,
184  button.insyde::after {
185    content: ' *';
186  }
```

Figure 5.2-1 Insyde appended

5.3 Enable Menu Style for One-Time Use at Runtime

If you don't want to recompile the program and just want to see the effect, you can modify the demo value stored in Session Storage.

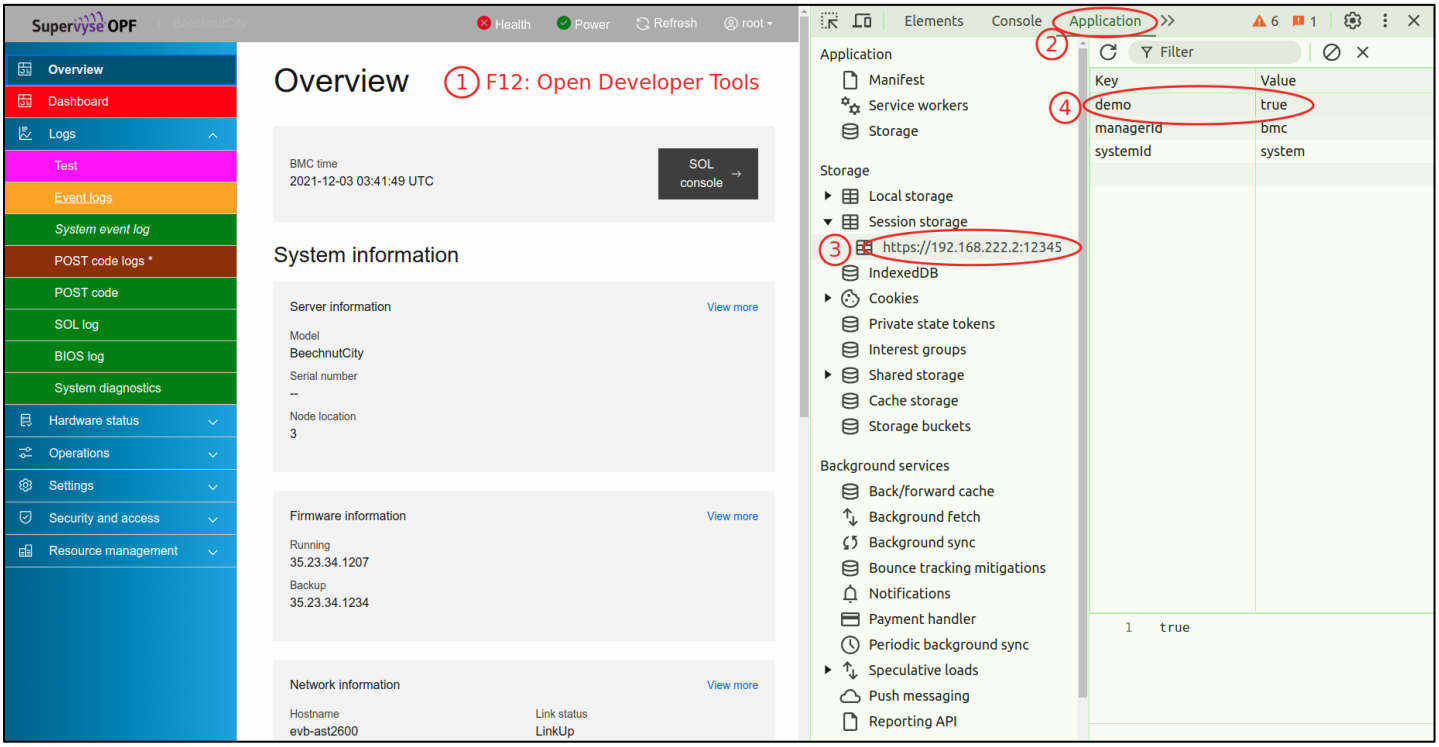

Figure 5.3-1 Web developer tools - runtime test

1. Press F12 on your keyboard to open the browser's "Developer Tools".
2. Select the "Application" tab.
3. Choose "Session storage" from the left-hand menu.
4. Variables are listed in the key/value format on the right side.
   Toggle the demo variable (true or false) and press F5 (browser reload) to apply.
   This value will reset to default when the page is closed and reopened.

5.4 Enable Normal/Demo Mode Toggling

To switch between the normal mode (left) and demo mode (right) menu styles without recompiling the code, you can use the following method: double-click the Insyde logo on the login page to toggle between the two modes.

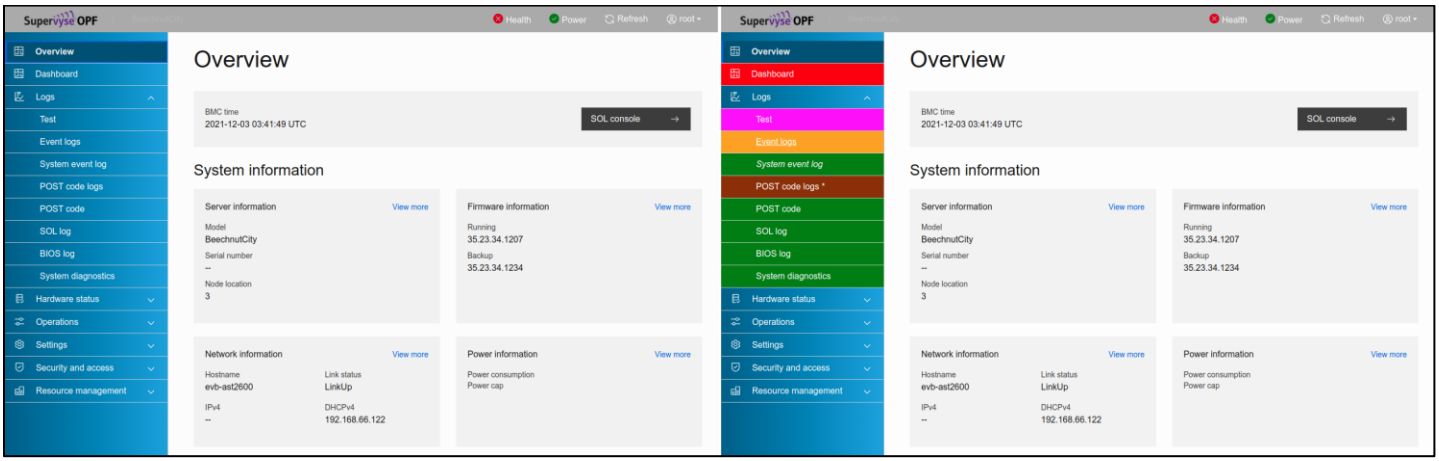

Figure 5.4-1 Normal/demo mode toggling

To enable this feature, first enable the corresponding option in menuconfig, save your changes, and then proceed with the image build process. After completing these steps, you will be able to switch modes at any time without the need to recompile.

Steps to enable it:

1. Run the command: bitbake pkconfig -c menuconfig
2. Select the corresponding option in the menuconfig.

```
(Top) → Network Services → Web Service → Web parameters
                                  Main menu
(128) User name max length on BMC web
(128) Password max length on BMC web
(180) Web heartbeat timeout
(10) Web heartbeat interval
(0) Timeout time of IKVM session connection
(30) Timeout time of IKVM socket connection
[ ] BMC Setting Preserve Option
[ ] Web page demo toggle

[Space/Enter] Toggle/enter  [ESC] Leave menu           [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Figure 5.4-2 Enable normal/demo mode toggling

After building the image and updating it to the BMC, double-click the Insyde logo on the login page to toggle to demo mode.

Check the demo mode flags by using the browser's developer console.

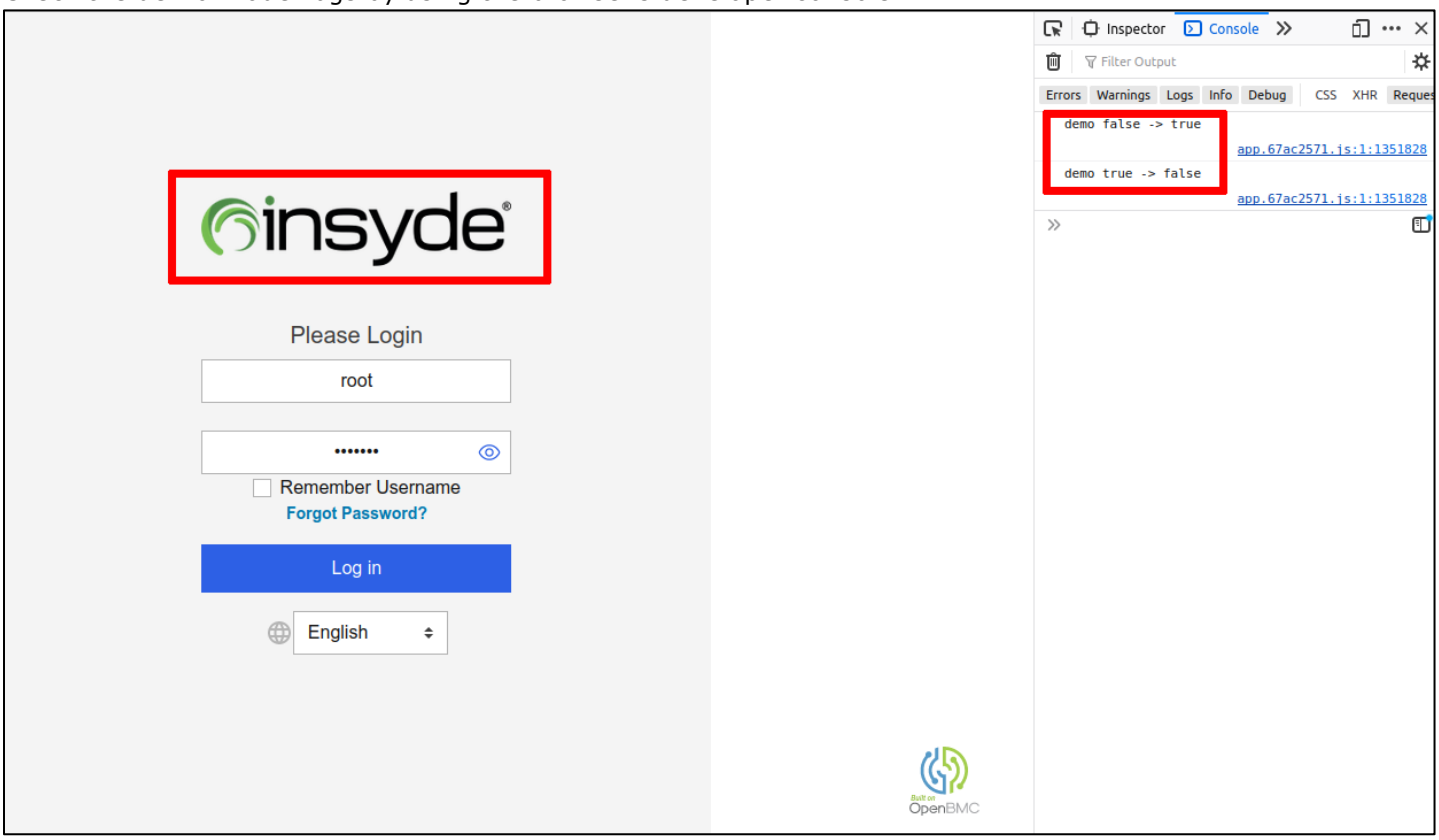

Figure 5.4- Toggling test

6. Develop and Debug

6.1 Develop with the Vue Dev Server

The **Vue dev server** refers to the development server used by Vue.js applications. When you set up a Vue project, the development server is automatically included as part of the Vue CLI (Command Line Interface). It serves your application, provides live reloading, and allows for quick development iteration.

Key Features of Vue Dev Server:

- **Hot Module Replacement (HMR)**: Updates the modules without a full page reload.
- **Live Reload**: Automatically reloads the page when changes are detected.
- **Proxy Support**: Useful for handling API requests while developing locally.
- **Customizable**: Port, proxy, and other settings can be configured in the vue.config.js file.

6.1.1 Install the Prerequisites

Before Vue dev server, make sure Node.js and npm are installed, as they are required for running build tools and managing packages.

```
# Install NVM (Node Version Manager)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
# Note: You need to log out and log back in after installation.

# Verify NVM installation
nvm –v

# Install a specific Node.js version
nvm install 18.16.0

# Verify Node.js and npm versions
node -v
npm -v
```

6.1.2 Configure Vue Dev Server

Add or edit the file located at: isc-src/web/webui-vue/.env.insyde.local

```
# Set the environment to development
NODE_ENV=development
# Backend server URL
BASE_URL=https://192.168.11.106:443
# Port for the localhost
VUE_APP_PORT=12345
# Debug level for logging
VUE_APP_LOG_LEVEL=debug
```

Field Description:

- **NODE_ENV**: Set this to development to enable development mode.
- **BASE_URL**: Set the proxy URL to forward frontend requests to the backend server.
- **VUE_APP_PORT**: Set the port for the development server to run on. The default is 8000.

- **VUE_APP_LOG_LEVEL**: Set the level of detail for the log output. Available options include silent, error, warn, info, and debug.

6.1.3 Run the Vue Dev Server

First, navigate to the isc-src/web/webui-vue directory.

Next, execute the following command:

```
npm install
npm run serve:insyde
```

Here's an explanation of the two commands:

- **npm install**
  This command is used to install all the dependencies listed in your project's package.json file. When you run this command, npm (Node Package Manager) will:
  - Download all the required packages and libraries that your project depends on.
  - Create a node_modules directory where these packages will be stored.
  - Create a package-lock.json (if it doesn't already exist), which locks the versions of the installed packages to ensure consistency across different environments.
- **npm run serve:insyde**
  This is a custom command defined in the scripts section of the package.json file of your project. The npm run command allows you to run any custom scripts that are defined in the scripts section. In this case, serve:insyde refers to a development server configuration specifically tailored for Insyde-related settings or features.

In Summary:

- **npm install**: Installs all the dependencies needed for your project to run.
- **npm run serve:insyde**: Runs the development server with a custom configuration, likely related to Insyde-specific features.

Upon successful execution, the following screen will be displayed.

```
DONE  Compiled successfully in 10946ms
                    10:03:08 AM


App running at:
- Local:   https://localhost:12345/
- Network: https://192.168.222.11:12345/

Note that the development build is not optimized.
To create a production build, run npm run build.
```

Figure 6.1.2-1 Run dev server

Connect to the provided IP address and port.

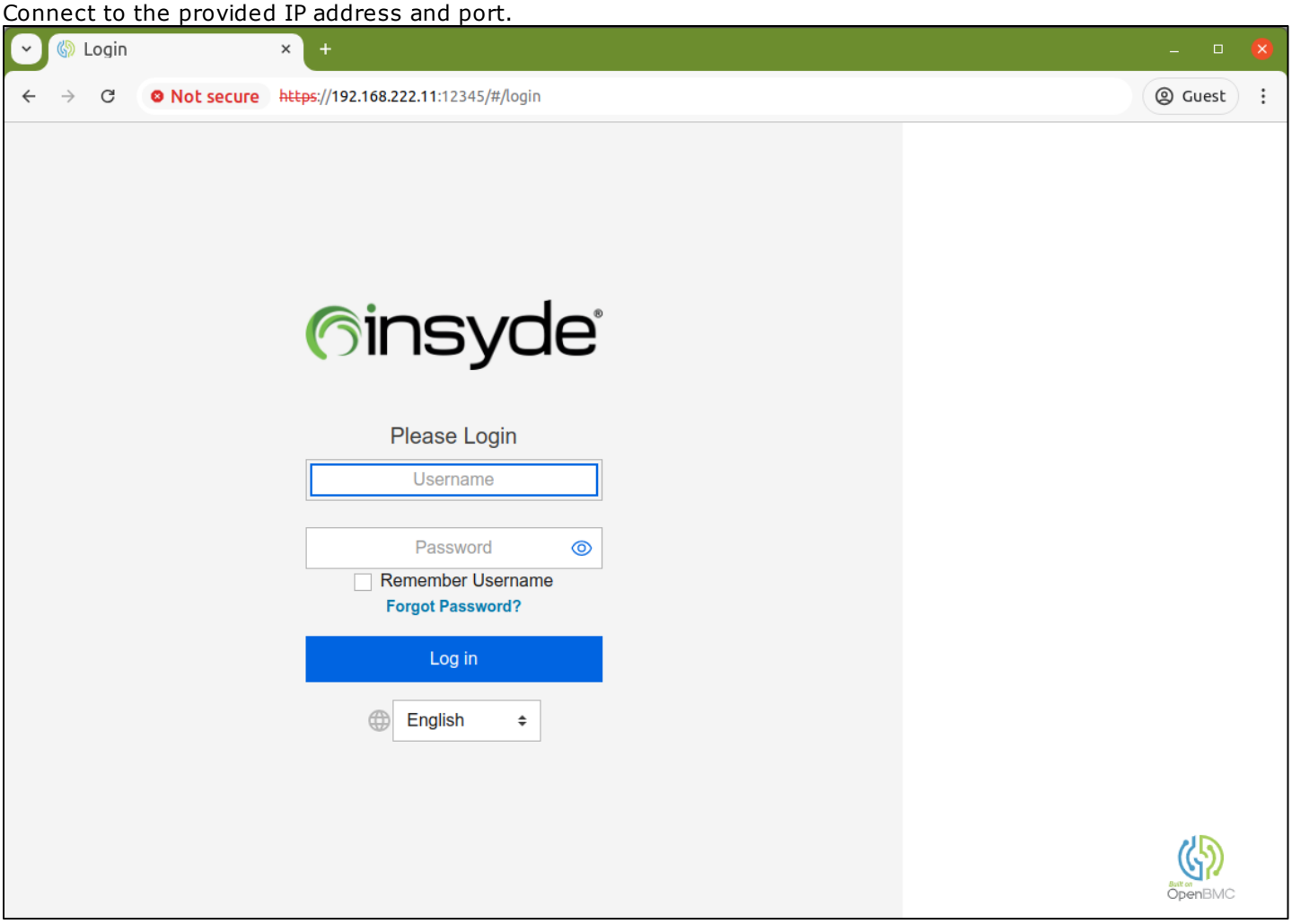

Figure 6.1.2-2 Connect to the web interface through the dev server

6.2 Debug with Vue.js Devtools

Vue.js Devtools is a powerful browser extension for debugging and inspecting Vue.js applications. It provides developers with an easy way to interact with and analyze the internal state of their Vue components.

Here's a brief introduction to its key features:

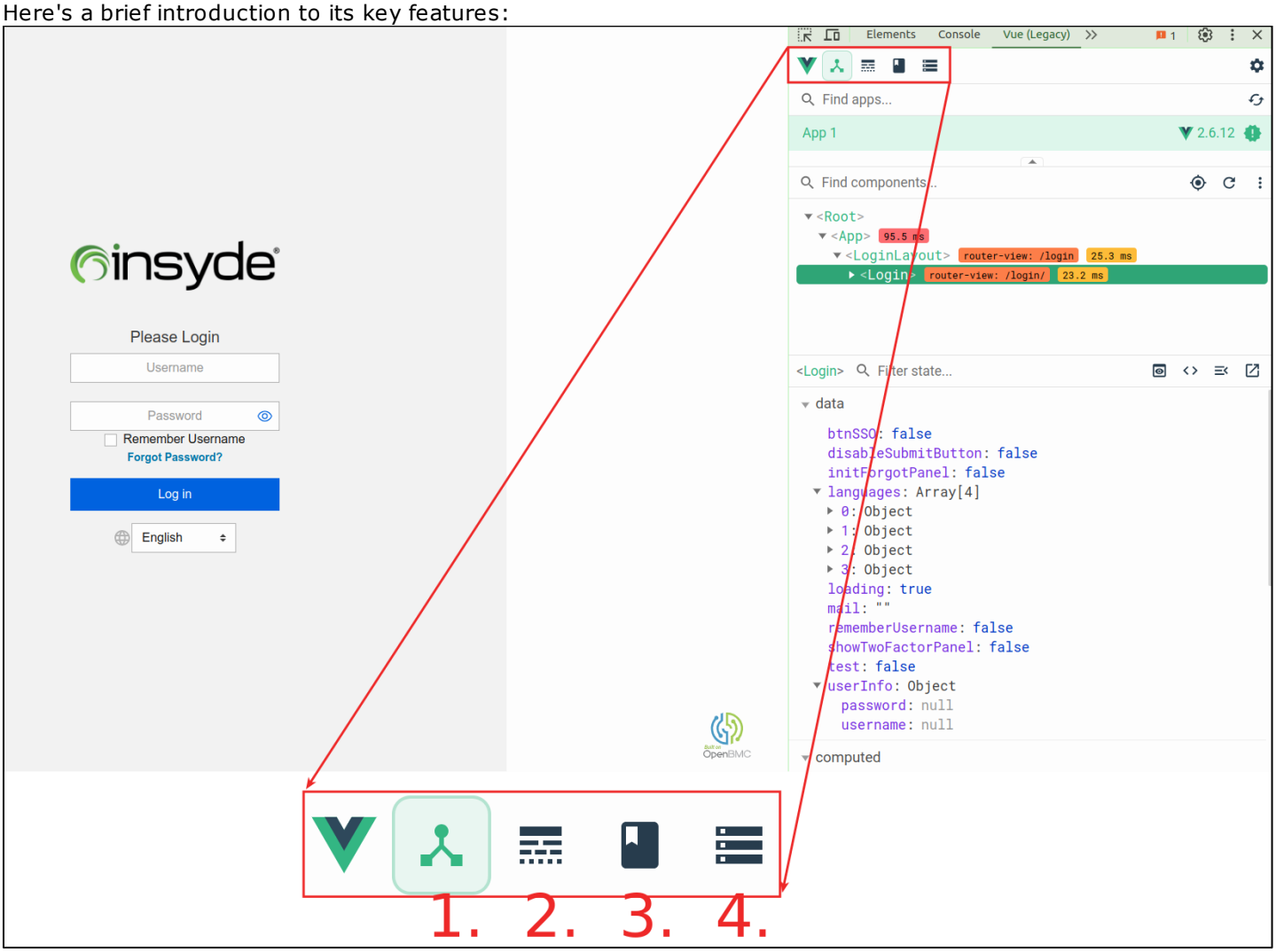

Figure 6.2-1 Vue.js Devtools

1. **Components**: Allows you to inspect and modify the component tree, props, data, and computed properties.
2. **Timeline**: Provides a timeline of Vue component activities, such as data changes, reactivity updates, and events.
3. **Routes**: Lets you inspect routing-related data in Vue Router, showing the current route, and route params.
4. **Vuex**: Allows you to inspect and interact with Vuex state, and getters for easier state management debugging.

Vue.js Devtools require the Vue page to be in **development mode**.

If your Vue app is in production mode, the devtools may not detect it. You can ensure your app is in development mode by setting the NODE_ENV environment variable to development during the build process. For example:

- Add or edit the file located at: isc-src/web/webui-vue/.env.insyde.local

```
# Set the environment to development
NODE_ENV=development
```

- Rebuild the image

```
bitbake webui-vue -c clean
bitbake webui-vue
bitbake <image-recipe>
```

6.2.1 How to Use Vue.js Devtools

Install the Vue.js Devtools extension from the Chrome Web Store or Firefox Add-ons.

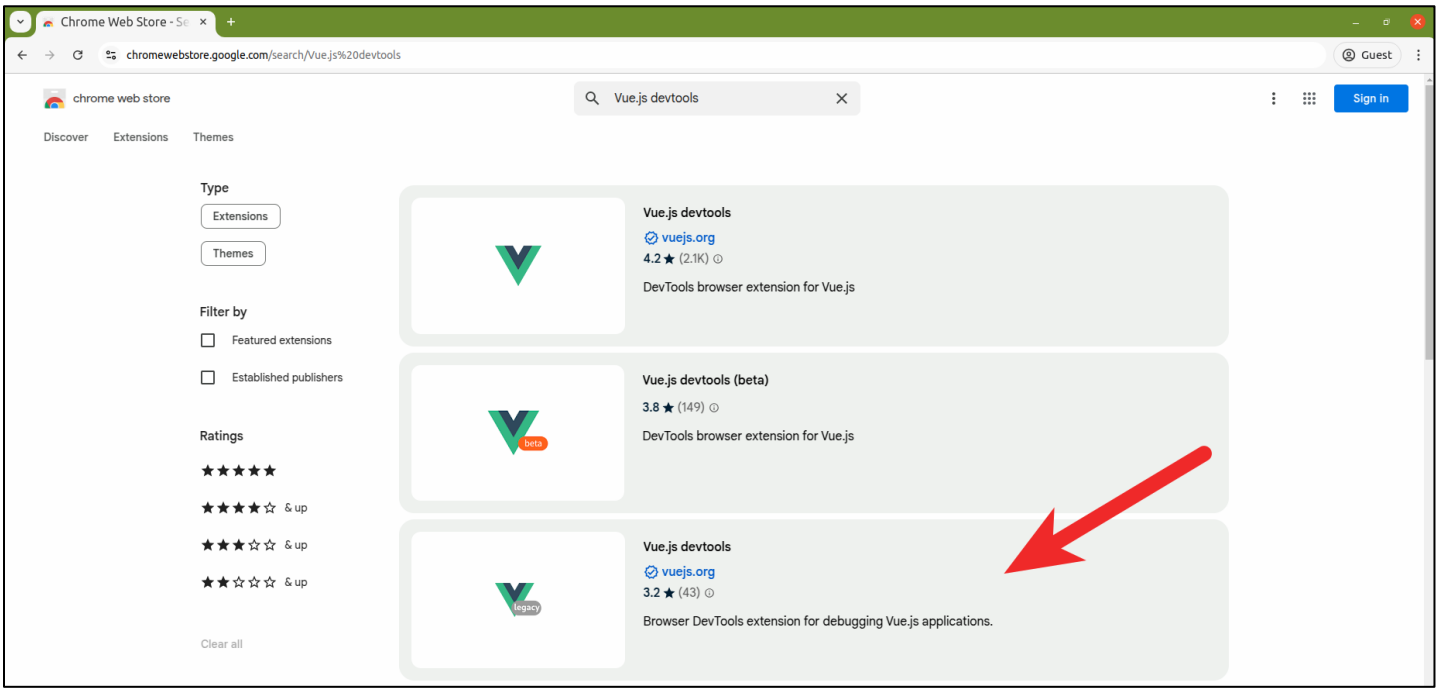

Figure 6.2.1-1 Install the Vue.js Devtools extension

Example:

1. Go to: https://chromewebstore.google.com
2. In the search bar, type "Vue.js devtools".
3. Look for the "Vue.js devtools" extension that supports Vue 2.x (this is the legacy version).
4. Click Add to Chrome to install the extension.

Once installed, you'll be able to use the Vue.js Devtools for debugging and inspecting your Vue 2.x applications.

6.2.2 Example to Inspect a Component and Vuex Store Using Vue.js Devtools

You can use the Vue.js Devtools to inspect a component by following these steps:

1. Press F12 on your keyboard to open the browser's Developer Tools.
2. Switch to the "Vue (Legacy)" tab.
3. Select the "Components" tab from the submenu.
4. Click on the desired component from the list displayed.
5. Inspect the component's data, props, computed properties.

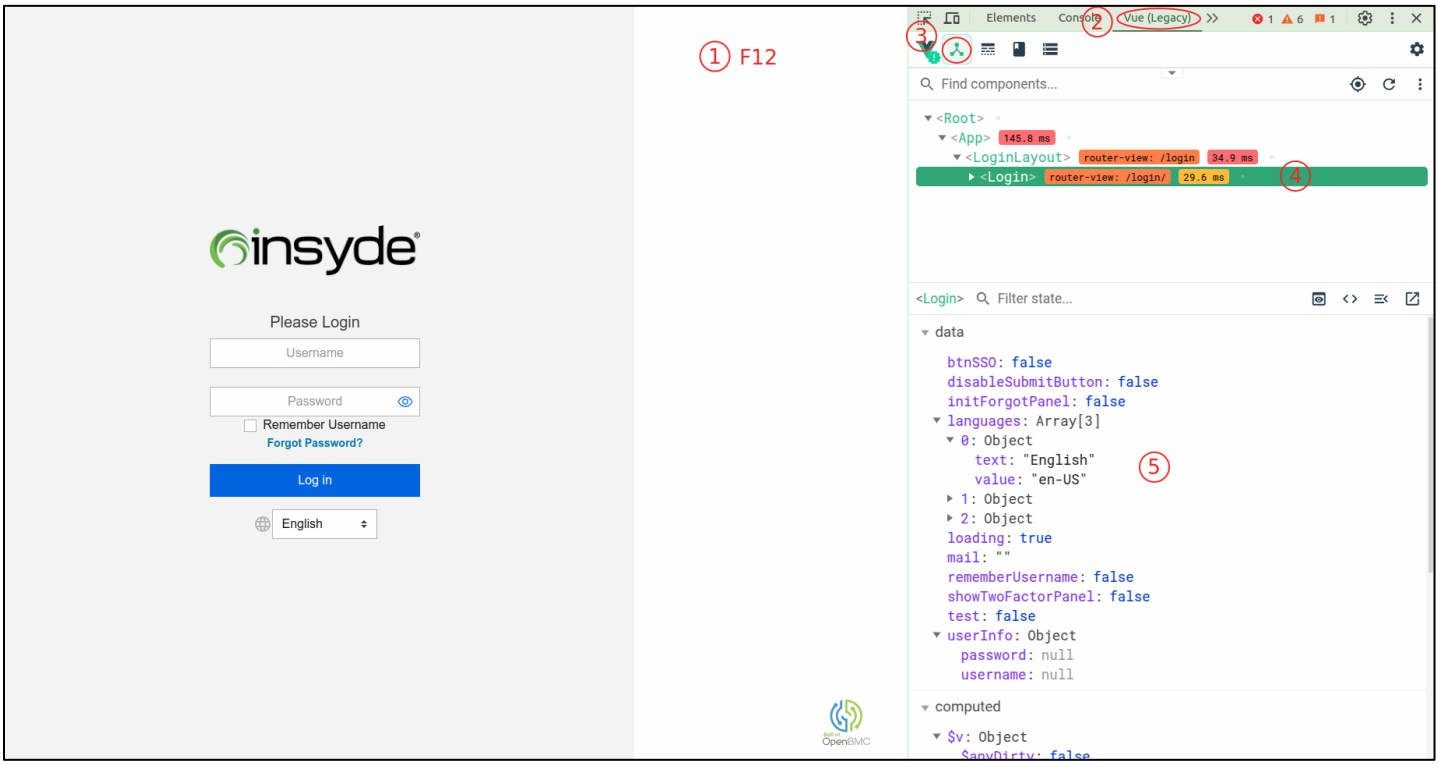

Figure 6.2.2-1 Use Vue.js Devtools to inspect a component

You can use the Vue.js Devtools to inspect a store by following these steps:

1. Press F12 on your keyboard to open the browser's Developer Tools.
2. Switch to the "Vue (Legacy)" tab.
3. Select the "Vuex" tab from the submenu.
4. Click on the desired store from the list displayed.
5. Inspect the store's state, and getters in the panel.

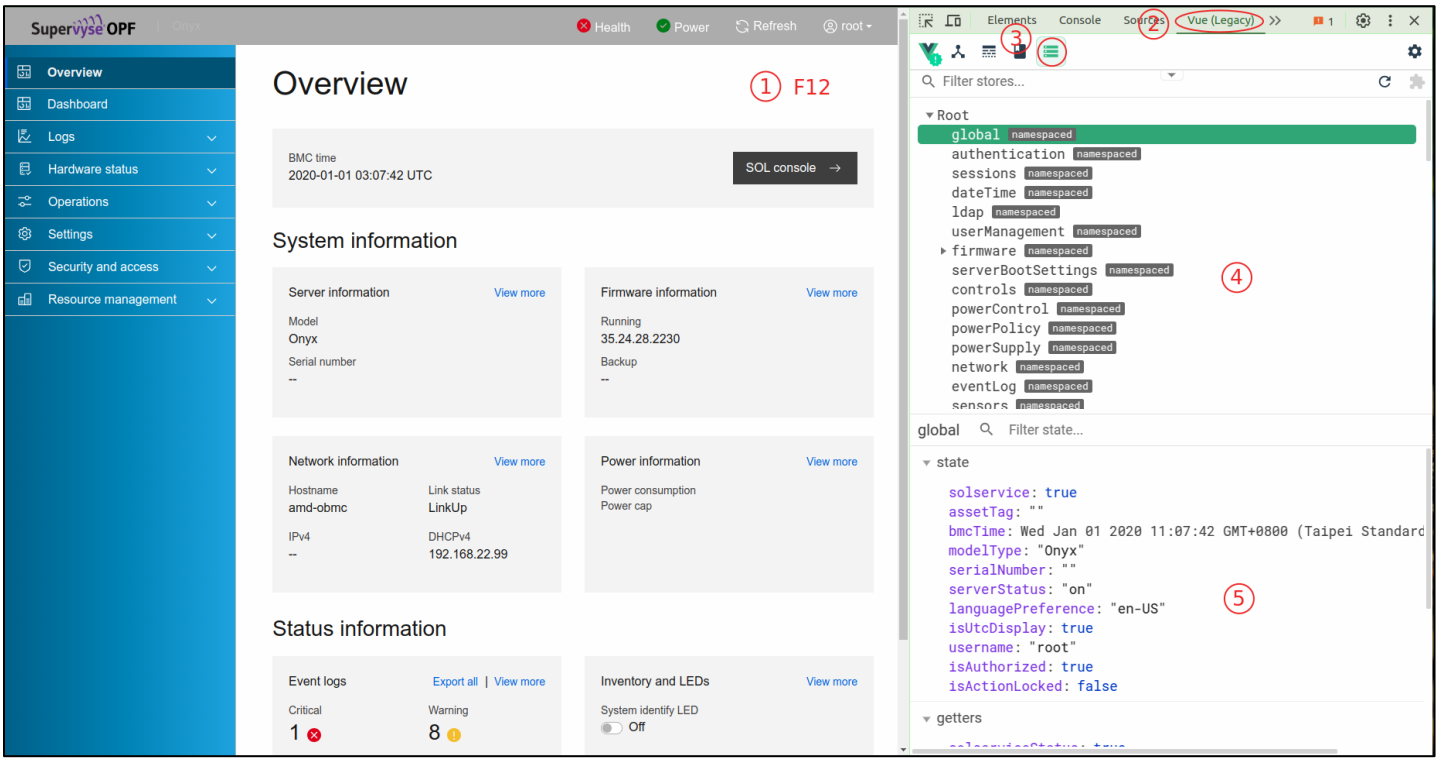

Figure 6.2.2-2 Use Vue.js Devtools to inspect a store

7. Vue with ESLint

Vue projects frequently incorporate ESLint to maintain code quality and adhere to best practices. ESLint serves as a powerful linter that checks JavaScript code, enforcing style rules, catching syntax issues, and helping to prevent bugs.

Key Benefits of Using ESLint with Vue

- **Enforces Coding Standards**: ESLint applies a set of predefined rules to ensure code follows team or industry standards.
- **Error Prevention**: Identifies syntax and logical errors, helping prevent common bugs.
- **Vue-Specific Rules**: With eslint-plugin-vue, ESLint provides Vue-specific rules that optimize .vue files, including templates, styles, and script sections.

By integrating ESLint with Vue, teams gain better control over code quality, making the development process smoother and the code easier to maintain.

7.1 How to Use ESLint Verify

In the ESLint CLI, you can use the eslint --fix command to check and automatically fix code issues, or eslint --no-fix to only validate without modifying code.

First, navigate to the isc-src/web/webui-vue directory and make sure you have installed the node_modules by running the command:

```
npm install
```

Next, execute the following command to check or fix the related code:

```
npx eslint <file.js|file.vue>        # Checks if the code meets the defined standards
npx eslint <file.js|file.vue> --fix  # Checks and automatically corrects applicable issues
```

Example:
When you encounter the following error while compiling or running the Vue Dev Server:

```
 WARNING  Compiled with 1 warning

 warning  in ./src/views/Logs/Test/Test.vue

Module Error (from ./node_modules/eslint-loader/index.js):

/home/jamey/src/OPF-Rev2.0/isc-src/web/webui-vue/src/views/Logs/Test/Test.vue
  12:11  error  Delete `·`  prettier/prettier

✖ 1 problem (1 error, 0 warnings)
  1 error and 0 warnings potentially fixable with the `--fix` option.


 @ ./src/views sync .*\.(js|vue)$
 @ ./src/env/router/insyde.js
 @ ./src/router/index.js
 @ ./src/main.js
 @ multi (webpack)-dev-server/client?https://192.168.222.11:12345&sockPath=/sockjs-node (webpack)/hot/dev-server.js ./src/main.js
```

Figure 7.1-1 ESLint errors

Run the following command to resolve it:

```
npx eslint src/views/Logs/Test/Test.vue --fix
```

This will automatically apply the necessary fixes to the specified file.

7.2 How to Config ESLint Rules

Location: isc-src/web/webui-vue/.eslintrc

Configure ESLint Rules: The .eslintrc file created will include rules for verifying code quality. You can modify the rules section to meet your project's coding standards. For example:

```
module.exports = {
  root: true,
  env: {
    node: true,
  },
  globals: {
    _: 'readonly',
  },
  extends: ['plugin:vue/recommended', 'eslint:recommended', '@vue/prettier'],
  rules: {
    'no-console': 'off',
    'no-debugger': process.env.NODE_ENV === 'production' ? 'error' : 'off',
    'prettier/prettier': [
      'error',
      {
        singleQuote: true,
      },
    ],
    'vue/component-name-in-template-casing': ['error', 'kebab-case'],
  },
  parserOptions: {
    parser: 'babel-eslint',
  },
  overrides: [
    {
      files: [
        '**/__tests__/*.{j,t}s?(x)',
        '**/tests/unit/**/*.spec.{j,t}s?(x)',
      ],
      env: {
        jest: true,
      },
    },
  ],
};
```

In this configuration:

- **root: true**
  root: true tells ESLint that this is the root configuration file, meaning ESLint will not look for configuration files higher up in the directory tree.
- **env: { node: true }**
  The env property defines the environments in which the code is expected to run. Setting node: true means

that this project is expected to run in a Node.js environment. This enables ESLint to recognize global variables and methods specific to Node.js, such as require, module, etc.

- **globals: { _: 'readonly' }**
  The globals property defines global variables. In this case, the _ variable (which typically refers to the Lodash library) is set as readonly, meaning the value of _ cannot be modified.
- **extends: ['plugin/recommended', 'eslint', '@vue/prettier']**
  The extends property specifies which shared configuration rules ESLint should extend:
  - plugin:vue/recommended: This is a set of recommended ESLint rules for Vue.js projects.
  - eslint:recommended: This is ESLint's built-in recommended rule set, which includes common checks.
  - @vue/prettier: This rule set integrates Prettier with ESLint to ensure that code formatting doesn't conflict with ESLint rules.
- **rules**
  The rules property defines specific ESLint rules:
  - 'no-console': 'off': This rule disables warnings for the use of console. By setting it to 'off', it allows console statements in the code.
  - 'no-debugger': process.env.NODE_ENV === 'production' ? 'error' : 'off': In a production environment (NODE_ENV === 'production'), the use of debugger will result in an error. In non-production environments, it's disabled ('off').
  - 'prettier/prettier': ['error', { singleQuote: true }]: This rule enforces Prettier code formatting with the singleQuote option set to true, meaning it prefers single quotes in the code.
  - 'vue/component-name-in-template-casing': ['error', 'kebab-case']: This rule ensures that Vue component names in templates must be in kebab-case (e.g., my-component). If violated, it will throw an error.
- **parserOptions: { parser: 'babel-eslint' }**
  parserOptions specifies the parser options. In this case, babel-eslint is used as the parser, which allows ESLint to understand the latest ECMAScript features (such as ES6, JSX, etc.).
- **overrides**
  The overrides property allows you to apply different ESLint rules to specific files. Here, the rules are applied specifically to test files:
  - files: ['**/__tests__/*.{j,t}s?(x)', '**/tests/unit/**/*.spec.{j,t}s?(x)']: This specifies the pattern for test files located in __tests__ and tests/unit directories with file extensions .spec.js, .test.js, etc.
  - env: { jest: true }: For these test files, the jest environment is enabled, which makes Jest global variables and functions available.

7.3 How to Ignore ESLint Rules

Location: isc-src/web/webui-vue/.eslintignore

The .eslintignore file is a configuration file for ESLint that tells ESLint which files or directories to ignore during the code linting process. This allows developers to skip files that don't need to be checked, such as third-party libraries, generated files, or test files.

Basic Syntax

The syntax of .eslintignore is similar to .gitignore and supports wildcards to specify files, directories, or paths to be ignored. Common configurations include:

- **File or Directory Names**: You can directly specify a file or directory name.
- **Wildcards**: Use * or ** to match multiple files or subdirectories.
- **Comments**: You can add comments with # to explain certain rules or directories.
- **Excluding Specific File Types**: You can exclude specific file types by their extensions.

Example:

Here are some examples of .eslintignore configurations:

```
# Ignore all .min.js files in the root directory
*.min.js

# Ignore the node_modules directory
node_modules/

# Ignore the dist directory, usually for compiled files
dist/

# Ignore all .spec.js files in the tests folder
**/tests/**/*.spec.js

# Ignore a specific file
src/legacyFile.js

# Ignore all .html files in the src directory
src/*.html

# Ignore a specific file in the root directory
debug.log
```

How It Works

- **Ignoring Directories**: Directories like node_modules/ and dist/ typically contain compiled or external dependencies that don't need linting.
- **Ignoring Specific File Types**: Files like .min.js or .html might not need to be frequently checked or are already minified, so they are typically ignored.
- **Ignoring Test Files**: Test files often have their own style guide, and developers may choose to ignore linting on them, especially if the test framework has its own linting rules.

Summary

The .eslintignore file is a simple and effective way for developers to specify which files or directories should not be linted, improving ESLint's efficiency and avoiding unnecessary error reports.

8. Use Browserslist to Support Specific Browser Versions

Browserslist is a tool that allows you to define the target browsers for your application in a shared configuration. This is particularly useful in Vue.js projects to ensure compatibility with specific browsers while optimizing the size and performance of the application.

Setting Up Browserslist in Vue.js

Step 1: Create a Browserslist Configuration
Browserslist configuration can be added to your project of the following way:
In the .browserslistrc file located in the isc-src/web/webui-vue folder.

Step 2: Define Target Browsers
Here are examples of how to define target browsers:

```
> 1%                # Supports browsers used by more than 1% of global users.
last 2 versions     # Supports the last two major versions of all browsers.
not dead            # Excludes browsers that are no longer maintained.
edge 113            # Explicitly includes Edge 113.
```

Use the following command to update the browser data used by Browserslist in your project. This ensures your project uses the latest browser statistics and versions for compatibility checks.

```
npx borwserslist@latest --update-db
```

Notice: Before executing the npx command, ensure that you first run `npm install' to install the necessary dependencies.

Use the command to verify which browsers are covered by your configuration.

```
npx browserslist
```

You can also go to browsersl.ist, paste the conditions, and check which browser versions are actually supported.

The following screenshot shows the result of the command `npx browserslist' without updating the browser data. You can try running it and compare the differences after updating the browser data.

```
$ npx browserslist
Browserslist: caniuse-lite is outdated. Please run:
npx browserslist@latest --update-db

Why you should do it regularly:
https://github.com/browserslist/browserslist#browsers-data-updating
and_chr 89
and_ff 86
and_qq 10.4
and_uc 12.12
android 89
baidu 7.12
chrome 89
chrome 88
chrome 87
edge 89
edge 88
firefox 86
firefox 85
firefox 78
ie 11
ios_saf 14.0-14.5
ios_saf 13.4-13.7
kaios 2.5
op_mini all
op_mob 62
opera 73
opera 72
safari 14
safari 13.1
samsung 13.0
samsung 12.0
```

Figure 8-1 Browsers support list